



Engineering Degree Project

PDF Parsing, Unveiling the Most Efficient Method



Author: Per Ingemarsson & Daniel Persson

Supervisor: Henrik Leion

Lnu Supervisor: Fredrik Ahlgren

Semester: Spring 2024

Subject: Computer Science

Abstract

This report investigates the challenges and methods associated with parsing PDF documents for use with Large Language Models (LLMs). The variability and complexity of PDF formats pose significant challenges in ensuring accurate data extraction and interpretation. We evaluate several parsing techniques, including rule-based, deep learning-based, and multimodal methods, to determine their effectiveness in handling diverse PDF content. Our study reveals that while multimodal methods generally outperform others, particularly in managing mixed content formats. For PDFs containing complex images a deep learning approach is more effective. This research offers valuable insights into optimizing PDF parsing strategies, balancing accuracy and cost-efficiency, thereby advancing the utility of LLMs in document processing and contributing to improved data management practices in various industries.

Keywords: PDF parsing, Large Language Models, data extraction, Multimodal method, deep learning, document processing

Preface

We would like to extend our gratitude to Tietoevry for their hospitality and for allowing us to work on this project at their office. Their support, including the much-appreciated coffee, was invaluable to the completion of this project. We also wish to express our deepest appreciation to our supervisor, Henrik Leion, and our Lnu Supervisor, Fredrik Ahlgren. Their combined guidance, expertise, and feedback were instrumental in shaping the direction and quality of our work

Contents

1	Introduction	1
1.1	Background	1
1.2	Related work	3
1.3	Problem formulation	4
1.4	Motivation	5
1.5	Milestones	5
1.6	Scope/Limitation	5
1.7	Target group	6
1.8	Outline	6
2	Theory	7
2.1	PDFs	7
2.2	Large Language Models	7
2.2.1	GPT-3.5-Turbo	8
2.2.2	GPT-4-Turbo	8
2.2.3	GPT-4o	9
2.2.4	LLaMa3 8B and 70B	9
2.2.5	Mixtral8x22B	9
2.2.6	Tokens and cost for LLMs	10
2.3	Retrieval Augmented Generation	11
2.4	OpenRouter	11
2.5	PDF Parsing Methods	12
2.5.1	Rule-Based Method	12
2.5.2	Deep Learning-Based Method	12
2.5.3	Multimodal	13
2.6	PDF Parsing Tools	13
2.6.1	PyPDF	13
2.6.2	PyMuPDF	14
2.6.3	Unstructured	14
2.6.4	Pdfminer.six	14
2.6.5	LLama Parse	14
2.6.6	Llama2 aided Tesseract	15
3	Method	16
3.1	Research Project	16
3.2	Method	16
3.3	Reliability and Validity	18
3.3.1	Reliability	18
3.3.2	Validity	19
3.4	Ethical considerations	19
3.5	Team Work	19
4	Implementation	20
4.1	Hardware	20
4.2	Software	20
4.2.1	Parsing PDFs	20
4.2.2	LLM prompting	21
4.2.3	AI scoring of the answers	21

4.2.4	Visualising of the results	22
4.2.5	Libraries	22
4.3	Answer grading	22
4.3.1	Prompt engineering for LLM grading	22
4.3.2	Evaluating manually grading vs LLM grading	23
4.3.3	The PDFs	23
5	Experimental Setup and Results	24
5.1	Experimental Setup	24
5.2	Results	24
6	Analysis	47
6.1	Overall performance of the LLMs	47
6.2	Overall performance of the Parsing Tools/Methods	47
6.3	Performance on All PDFs	48
6.4	Performance on AidonHanInterface PDF	48
6.5	Performance on Images PDF	49
6.6	Performance on Mixed1 PDF	49
6.7	Performance on Tables PDF	50
6.8	Performance on Texts PDF	50
6.9	Prompt Cost	51
6.10	Questions on Aidonhaninterface PDF	51
6.11	Questions on Images PDF	51
6.12	Questions on Mixed1 PDF	52
6.13	Questions on Tables PDF	52
6.14	Questions on Texts PDF	52
6.15	Conclusion of Analysis	53
7	Discussion	54
7.1	RQ 1	54
7.2	RQ 2	55
7.3	RQ 3	55
7.4	Related Works	56
7.5	Observations	56
7.6	Larger Scaled Test	57
7.7	Comparing Cost of Parsing Tools	57
7.8	Evolution of LLMs	58
8	Conclusion	59
8.1	Future work	59
	References	61
A	Appendix 1	A

1 Introduction

This chapter presents the foundation for this degree project, which focuses on utilizing PDF Parsing Tools to parse documents for use with Large Language Models. It highlights the challenge organizations face in efficiently managing data in today's digital age. It aims to evaluate different methods and models for parsing PDFs, taking into account factors like complexity and cost. Additionally, it also touch on related works and pose clear research questions to guide the investigation. The motivation stems from the broader implications of this project, not only for Tietoevry but also for the wider scientific community. Finally, it provide a brief overview of the project's milestones, scope and target audience.

1.1 Background

Organizations have faced the challenge of harnessing valuable insights from data long before the digital era. This historical struggle shows the need to interpret and utilize information effectively. The introduction of computers and advanced analytics has transformed this challenge, offering tools to navigate and utilize the data. But still, there are challenges to effectively using all the digital data. One of the primary issues is managing the enormous volumes of data generated in the digital era. The sheer amount of data makes it difficult for organizations to filter and extract the most relevant insights. This challenge is made more difficult by the fast pace of data creation and the wide array of data types, including both structured and unstructured formats [1].

A helpful tool to handle this challenge is Artificial Intelligence (AI), AI offers organizations powerful solutions for managing large volumes of data with speed and efficiency. Using AI technologies, organizations can perform a thorough and accurate data analysis, uncovering valuable insights that could otherwise remain hidden. The ability of AI to detect patterns and anomalies enables a deeper understanding of the data, contributing to informed decision-making. This capability not only reduces manual workload but also streamlines data management processes [2].

In recent years Generative Artificial Intelligence has become popular, a well known Generative AI is ChatGPT. It operates as a language model developed by OpenAI, employing a machine learning architecture known as transformers. These transformers are designed to handle and predict sequences of data, particularly text, by capturing the complexities of language through what are known as attention mechanisms. These mechanisms enable the model to weigh the importance of different words in a sentence, thereby facilitating a deeper understanding of linguistic context and nuances. Since its creation, the development of ChatGPT has been marked by substantial evolution. Early language models were relatively small, both in terms of the data they were trained on and their computational capabilities. They could perform simple tasks like completing sentences or generating responses based on limited prompts. However, these early models often struggled with maintaining context or producing coherent long-form content [3].

As advancements in hardware and machine learning techniques progressed, so did the scale of language models. Modern versions, such as GPT-3, represent a significant leap, containing billions of parameters which allow them to process vast amounts of text and learn from a diverse array of internet-based sources. This scalability has not only enhanced their linguistic accuracy, but also their ability to have more sophisticated dialogues

and understand complex requests. The transition from smaller to larger models has been driven by the need for models that can generate more contextually accurate and nuanced text. Larger models are capable of storing and referencing more information, thereby improving their ability to understand and generate responses that are more relevant and diverse. However, this increase in size and complexity also presents challenges, including the need for greater computational resources and more sophisticated strategies to manage potential biases in training data [3].

By utilizing Large Language Models (LLMs) like ChatGPT, which can understand and generate human-like text, it enables organizations to extract valuable insights from unstructured data and documents, streamlining operations and reducing manual effort. These models not only open new ways to handle vast amounts of data but also introduce challenges on how to effectively use these tools. A method called retrieval augmented generation (RAG) allows LLMs to access and use external data, enhancing their ability to answer questions and making the data more accessible and usable for organizations [4, 5].

However, managing the diverse data formats in PDFs still presents significant challenges for LLMs. PDFs often contain a mix of text, tables, images, and non-standard formatting, complicating their processing. This shows the need for the development of advanced methods capable of accurately interpreting and extracting information from various layouts and formats. Addressing these complexities is important for improving the effectiveness of LLMs, enabling seamless integration and analysis of data from multiple sources. Extracting data from PDFs involves sophisticated techniques due to the variety of content types they can contain. For instance, text-based PDFs can be processed using tools like PyPDF or PDFMiner, while scanned or image-based PDFs require OCR tools such as Tesseract to convert images to text. Each type of PDF requires custom approaches to ensure accurate data extraction and processing [6].

It is also important to acknowledge the great energy consumption of LLMs, both during the training and the operation, that will have an impact on the global environment. Recognizing this shows the importance of optimizing the use of LLMs to minimize the environmental footprint. Also from an economic standpoint, organizations stand to gain from this with the reduced energy consumption [7].

Tietoevry worked on a project last year where they made a RAG prototype based on 5000 PDF documents, the results were good, but it took several hours to read all the information from these documents. When it was time to take the step from prototype to product, they realized that to be able to read in several million documents, the costs would far exceed the benefits, so the solution was effective but practically unusable making it inefficient. This year, they are working on a project with the exact same problem, it is easy to find good tools to load PDF documents, but without a methodology to determine if a tool is good enough, they can't estimate the cost. With unlimited resources it is of course no problem to solve anything but in reality resources are limited and the challenge is to find the breaking point when a tool is no longer practically useful.

By investigating different methods for parsing PDF files for use with a LLMs, our research contributes to the ongoing evolution of natural language processing and information management. This, in turn, promises to positively impact society by facilitating easier access to and more efficient handling of text-based information.

1.2 Related work

In the field of PDF parsing, several related works have explored various methods and tools to improve the extraction and interpretation of data from complex PDF documents. This section reviews related studies that have contributed to advancements in PDF parsing techniques and their integration with LLMs.

Demiao Lin investigates PDF parsing accuracy on the effectiveness of professional knowledge based question answering (QA) systems. It highlights the limitations of current methods relying on high-quality text corpora and the significance of accurate PDF parsing for accessing professional documents. Through empirical experiments using the RAG technique, the study demonstrates that ChatDOC equipped with an advanced PDF parser, outperforms baseline systems on a significant portion of questions, showcasing the potential for revolutionizing RAG through improved PDF structure recognition [8].

Naihao Deng et al conducted a research on the effectiveness of different multimodal LLMs in interpreting tabular data using different prompting strategies and data formats. The analysis extends across six benchmarks covering tasks like question answering and fact-checking related to tables. Notably, it introduces the evaluation of LLMs' performance on image-based table representations. This study compares five text-based and three image-based table representations, showcasing how different representations and prompting strategies influence LLM performance. The research provides insights into the effective utilization of LLMs for tasks involving tables. Furthermore, the study also compares the performance of the top-performing open-source LLM (LLaMa-2-70B) with a closed-source counterpart (GPT-4), this showcases that the closed-source LLMs outperforms the open-source LLMs [9].

The "Use of LLM for Methods of Information Retrieval" study by University of Twente researchers explores using LLMs for efficient data retrieval from large data collections like PDFs and text files. Presenting a prototype, the paper showcases LLMs' potential to transform traditional search methods by using natural language processing to comprehend and sort through vast data volumes quickly. This approach shows enhanced search efficiency while maintaining high data security standards. The system, designed on a client-server architecture, incorporates functionalities such as data ingestion and indexing, with the goal of simplifying information retrieval processes. Despite facing challenges like bias and the environmental impact of LLMs, the paper views these models as valuable assets for information retrieval tasks, suggesting areas for future improvements to refine the system's performance [10].

Lewis et al introduces an approach to enhancing the performance of LLMs on knowledge-intensive natural language processing (NLP) tasks using retrieval-augmented generation (RAG) models. These models combine the parametric knowledge embedded within pre-trained models with non-parametric information retrieved from a vector index of Wikipedia. The study contrasts two RAG model formulations: one that utilizes the same set of retrieved passages across the entire generated sequence, and another that allows for different passages to influence individual tokens within the output. This methodology was evaluated across a range of NLP tasks, demonstrating state-of-the-art performance on three

open-domain question-answering benchmarks, superior to both purely parametric models and traditional retrieve-and-extract systems. Additionally, RAG models showcased their ability to generate more factually accurate, specific, and diverse responses compared to a leading parametric-only baseline. The research highlights the benefits of integrating parametric and non-parametric memory systems for tasks requiring access to external knowledge [5].

Mikko Korpela delves into the complexities of data extraction from PDFs and presents an approach utilizing GPT-4 which is a multimodal model. The author discusses various tools like PyPDF and Camelot for text and tabular extraction, highlighting their respective strengths and limitations. Through a case study involving the extraction of data from a technical manual, the author demonstrates the effectiveness of the multimodal approach. However, it's noted that this method is time-consuming, with processing times averaging around a minute per page due to the execution of the OpenAI GPT-4 model. A comparison with LlamaParse showcases differences in table quality and image descriptions, highlighting the nuanced challenges in data extraction. In conclusion, while the multimodal model shows promise for data extraction, there are notable challenges such as result variability and the interpretative nature of LLMs, necessitating further exploration in future developments [11].

1.3 Problem formulation

The primary problem lies in ensuring that the PDF parsing tools accurately can parse the wide range of data that is presented to them. Secondary is to ensure that the LLM can comprehend and process the data provided by the parsing tools. This problem is made more complex when the data can have a variety of formats, which includes plain text, tables and images. Each approach to processing the data has its own unique set of strengths and weaknesses, which makes it challenging to determine the optimal approach for different types of data. Additionally, the specific strengths and weaknesses associated with different types of PDF data remain largely unexplored. This project aims to systematically assess various methods to uncover the benefits and limitations to the methods, thereby providing a more informed selection of techniques for handling diverse data formats.

- RQ 1, How do the different parsing tools/methods handle the varying complexity of the PDFs, what are the benefits and limitations of the methods?
- RQ 2, How well can the different LLMs understand the varying complexity of the parsed PDFs, what are the strengths and weaknesses of each model?
- RQ 3, How do the different parsing methods and LLMs compare in terms of cost-efficiency?

The expected result of this degree project is an evaluation and comparison of the PDF parsing tools/methods and LLMs for parsing PDF documents. This is a comparison across various types of PDFs, as well as an assessment their cost-efficiency. The aim is to identify which approaches not only perform best in accurately interpreting complex PDF formats but also offer economically viable solutions. Through this project, we intend to provide valuable insights into optimizing PDF processing and balancing performance with environmental and financial considerations.

1.4 Motivation

Addressing the issues identified in the problem formulation is important for gaining insights into PDF parsing. By addressing these challenges, TietoEvry can gain a deeper understanding of the most suitable methods, optimizing for both parsing-quality and cost-effectiveness across various data types. This understanding is valuable for navigating the diverse landscape of Parsing Methods and LLMs, assisting in the selection of models for specific contexts. The impact extends beyond TietoEvry, benefiting other companies and society. Given the rapid evolution of LLM technology, our study contributes to advancing science and the IT/data industry by enriching the knowledge of PDF parsing.

1.5 Milestones

The milestones for this project are presented in Table 1.1.

Table 1.1: The milestones for the project

M1	Explore the parsing methods
M2	Create the PDFs dataset
M3	Familiarize ourselves with the methods and LLMs
M4	Implement and test the methods and LLMs
M5	Conduct tests with the PDF files
M6	Evaluate and analyze the test results

1.6 Scope/Limitation

This project aims to assess the various methods used in parsing PDF documents into LLMs. The main focus of this analysis is on how these different approaches handle the different complexities and formats of PDF documents, aiming to determine their efficacy in enabling LLMs to accurately understand and interpret the data. The objective is to evaluate these methods and models to find their strengths and weaknesses. Additionally, this study will consider both the effectiveness and the cost-efficiency of the techniques used, providing a complete view of their practical applications. This analysis is intended to guide future improvements and optimizations in LLM technology for enhanced data processing and management.

In this project, although there are multiple ways to fine-tune the methods used, it is not feasible to test the fine-tuning of the different models due to the significant increase in workload and testing. This limitation means that the potential improvements achievable through fine-tuning have not been explored.

This project will use a select few LLMs. This limitation means that our findings might not be fully representative of the broader array of LLM technologies available. By concentrating on a few specific models, we aim to delve deeper into their specific functionalities and performance characteristics. However, this approach may limit the generalizability of our results, as different models could exhibit varied behaviors under similar conditions.

Another limitation is in the selection and number of PDF files used for testing, which varied in complexity from simple to complex. The first PDF contains only text for evaluating basic text extraction capabilities. Subsequent PDFs introduced an incremental rise

in complexity: the next included tables, followed by another with images, to assess the LLMs' ability to process diverse data types. The complexity of the following PDF combines text, images, and tables in a multifaceted layout. Importantly, all content within these test PDFs was limited to the English language.

1.7 Target group

This research aims to provide valuable insights into PDF parsing with LLMs, aiming to unveil the most effective parsing methods for specific tasks. The significance of this research extends beyond TietoEvy, including software engineers, society, and other IT companies.

The handling of large quantities of documents poses a significant challenge in today's digital landscape and with the fast evolution of LLMs, new challenges are created. As these models become more sophisticated and capable, there arises a pressing need to ascertain the optimal LLM approach for efficient PDF parsing. This knowledge not only benefits TietoEvy and other IT companies but is also relevant for software engineers tasked with developing document processing solutions. Moreover, the impacts on the society of this research are noteworthy. By identifying the most effective method for PDF parsing with LLMs, it helps to gain knowledge for more streamlined and cost-effective document management practices. This, in turn, can lead to reductions in environmental impact, as fewer resources are consumed in running multiple LLMs simultaneously.

In essence, this research seeks to address a multifaceted challenge and to provide knowledge for the stakeholders which will help them to handle document processing tasks more efficiently and sustainably.

1.8 Outline

Chapter 2: Theory - This chapter is intended to provide an overview of the topic, which is needed to understand the report. It delves into the theoretical foundations and concepts relevant to the research topic.

Chapter 3: Method - The research project and method are outlined, detailing how the investigation was conducted.

Chapter 4: Implementation - This chapter details hardware and software used, the methods employed for parsing PDFs, the prompting of LLMs, the scoring of the answers by AI, and the visualization of the results

Chapter 5: Experimental Setup and Results - This chapter details the hardware specifications, libraries, frameworks, and software configurations used. Also presents the results

Chapter 6: Analysis - This chapter delves into the results and analyses them.

Chapter 7: Discussion - In this chapter the research questions (RQs), observations and the broader implications of findings are explored

Chapter 8: Conclusion - The project is summarized and the future work is presented.

2 Theory

This chapter provides a comprehensive exploration of the theoretical framework underpinning the thesis implementation. It serves as a foundational resource for understanding the subsequent sections of the report, offering detailed explanations of key theories and concepts.

2.1 PDFs

PDF is an abbreviation that stands for Portable Document Format. It is a versatile file format created by Adobe in 1992. The goal was to enable anyone to capture documents from any application and send electronic versions of these documents anywhere to be able to view and print them on any machine. Today, the PDF format gives an easy and reliable way to present and exchange documents, regardless of the software, hardware or operating system being used. The PDF format is standardized as ISO 32000 and is an open standard trusted by businesses around the world [12].

While PDFs are widely used, extracting the content from them can be complex and challenging, they come in various layouts, including single-column, multi-column and complex structures. PDFs often contain complex structures like tables, columns, headers, and footers, disrupting the sequential flow of text and complicating accurate extraction while preserving the original formatting. Scanned or image-based PDFs pose a significant challenge as the text is embedded within images. OCR technology is necessary to convert these images into machine-readable text. However, errors may occur even with OCR, especially with handwritten text or low-quality scans. PDFs can also contain various fonts, encodings, and character sets, further complicating text extraction. Inconsistent font styles or non-standard character encodings may result in incorrect text rendering or missing characters. PDFs could also have multiple columns to present information, like newspaper layouts, making it challenging to extract text while maintaining the correct reading order. Headers and footers in PDF documents often contain metadata, page numbers, and document titles, posing difficulties in separating these elements from the main text during extraction [13].

2.2 Large Language Models

Large Language Models (LLMs) are AI systems designed to understand, interpret, and generate human language by mimicking the way humans use words. LLMs are a subset of foundation models, which means they provide basic capabilities that can be tailored to a variety of tasks without needing to train separate models for each task, making them more efficient and versatile. To work, LLMs leverage a neural network architecture called transformers. These transformers excel at managing sequential data, such as text, by using what is known as an attention mechanism. This mechanism enables the model to focus on different parts of the input data, which is essential for understanding context and meaning in language.

The training process for LLMs involves learning from a vast corpus of text data. They learn by predicting the next word in a sequence, which allows them to generate coherent and contextually appropriate text. This is done through tokenization, where text is broken down into smaller pieces (tokens) that are then converted into numerical values (embeddings) that the model can process. LLMs are trained with billions of parameters that help

them capture the complexities of language, such as grammar and syntax, as well as the subtle relationships between different concepts. This training empowers them to perform a wide array of tasks, including but not limited to translation, summarization, question-answering, and creative content generation.

The power of LLMs lies in their ability to generate human-like text and to perform language understanding tasks at a scale that was not previously possible. They have applications in various fields, such as customer service, content creation, and even software development [14].

2.2.1 GPT-3.5-Turbo

GPT-3.5 is an advanced closed-source and well-known text-only LLM from OpenAI, designed to handle complex language tasks with greater efficiency and customization. Utilizing a transformer architecture, it processes large chunks of text by understanding context within inputs up to 16,000 tokens via API. The model excels in fine-tuning for specific applications, such as customer service or technical support, enhancing performance in tailored scenarios. Developers can guide the model's outputs to align with specific instructions or style requirements, making it ideal for maintaining a consistent brand voice. GPT-3.5 Turbo also incorporates function calling capabilities, enabling it to perform specific actions like API calls directly from natural language instructions, simplifying complex tasks into actionable responses.

GPT-3.5 Turbo is significantly cheaper than GPT-4, facilitating broader and more sustained use in commercial and research environments. Fine-tuning capabilities allow developers to customize the model to specific needs, resulting in improved steerability, consistent output formatting, and a custom tone. This customization reduces the size of prompts, making API calls faster and more cost-effective. Available for free, it is accessible for a wide range of applications, from customer support to content creation. Safety is prioritized through rigorous moderation of training data, ensuring reliable and secure outputs [15, 16].

2.2.2 GPT-4-Turbo

GPT-4 is closed-source LLM developed by OpenAI and represents a significant advancement in the field of artificial intelligence, especially in natural language processing. It is a multimodal model, which means it can process both text and images to generate outputs. Despite its advanced capabilities, it's still not on par with human performance in many real-world scenarios, but it excels remarkably in various professional and academic benchmarks. GPT-4 is built using a transformer architecture which is crucial for handling complex language models. This architecture allows the model to better understand context and generate more coherent and contextually appropriate responses. The model has been trained on a diverse dataset, encompassing a vast range of internet text, which helps it mimic human-like text generation across various topics and formats.

One of the standout features of GPT-4 compared to its predecessors (such as GPT-3) is its ability to process significantly larger context inputs with up to 128,000 tokens via API. This opens new possibilities for applications like long-form content creation, extended conversations, and detailed document analysis. GPT-4 also integrates improvements in safety and alignment, making it less likely to produce responses that encourage harmful

content and more likely to generate factually correct outputs. These enhancements have been achieved through extensive training, including human feedback, and collaboration with experts across different fields to refine its performance and safety measures. Additionally, GPT-4 has shown its versatility by performing well on tasks that require visual inputs as well as text. It can analyze images and generate relevant text outputs, demonstrating a significant step forward in integrating different forms of data for comprehensive AI analysis and interaction [17, 18].

2.2.3 GPT-4o

Released on 13 May 2024, GPT-4o is OpenAI's latest closed-source flagship model and represents a significant advancement in multimodal AI capabilities. Unlike its predecessor GPT-4, GPT-4o integrates text, audio, and vision inputs and outputs into a single neural network, allowing for more natural human-computer interactions. This model can process and respond to audio inputs in real time, with latency comparable to human response times, and it performs exceptionally well in multilingual and vision-related tasks. Furthermore, GPT-4o is optimized for efficiency, offering faster and more cost-effective API access (50% cheaper) compared to GPT-4 Turbo. Its development emphasizes safety and robustness, incorporating extensive safety evaluations and external testing. [19]

2.2.4 LLaMa3 8B and 70B

Meta's LLaMa3 8B and 70B models are advanced open-source LLMs designed for a wide range of natural language processing tasks, offering exceptional efficiency and accuracy. These models utilize the Transformer architecture, which is known for its self-attention mechanism, allowing the model to focus on different parts of the input data to generate more contextually relevant outputs. The 8B and 70B parameter LLaMA 3 models represent a significant advancement over their LLaMA 2 predecessors, setting a new benchmark for LLMs at these scales. Due to enhancements in both pretraining and post-training processes, our pretrained and instruction-fine-tuned models are currently the best available at the 8B and 70B parameter scales. Notable improvements in post-training procedures have significantly reduced false refusal rates, enhanced alignment, and increased the diversity of model responses. Additionally, LLaMA 3 exhibits greatly improved capabilities in areas such as reasoning, code generation, and instruction following, making the model more steerable.

The LLaMa3 8B model is designed with 8 billion parameters. It's part of Meta's tiered approach to model sizes, which includes models ranging from 7b up to 65b parameters. This variety allows for different levels of complexity and processing needs, catering to various AI tasks from basic language understanding to more complex problem-solving. On the other hand, the LLaMa 3 70B model is significantly more powerful with 70 billion parameters. It is capable of more nuanced understanding and generation of text, making it suitable for more demanding AI applications. Both models are part of Meta's ongoing efforts to push the boundaries of what AI can achieve in natural language processing [20].

2.2.5 Mixtral8x22B

Mixtral 8x22B is a state-of-the-art open-source LLM developed by Mistral AI. It utilizes a Sparse Mixture of Experts (SMoE) architecture, which allows the model to optimize computational efficiency by selectively activating a subset of its parameters during inference.

Specifically, while Mixtral 8x22B has a total of 176 billion parameters, only 44 billion are active during any given computational step. This selective activation significantly reduces computational overhead and enhances performance, making it both powerful and cost-effective.

The model is notable for its large context window of 65,000 tokens, enabling it to handle extensive and complex text inputs effectively. Mixtral 8x22B demonstrates exceptional capabilities across various natural language processing tasks, including multilingual understanding, code generation, and mathematical reasoning. It outperforms many existing models, including LLaMA 2 70B, on standard benchmarks [21].

2.2.6 Tokens and cost for LLMs

Using and hosting LLMs incurs significant costs due to various factors involved in their deployment and operation. First, the use of LLMs involves computational expenses. These models require substantial processing power to function, typically provided by powerful GPUs (Graphics Processing Units) or TPUs (Tensor Processing Units). When you use an LLM via a cloud service, you pay for the computational resources on a per-usage basis. The cost scales with the complexity of the task, the size of the model, and the amount of data processed. Second, data storage and bandwidth costs add to the expense. LLMs, especially those used in real-time applications, require fast and reliable storage solutions to handle large volumes of data efficiently. Additionally, transmitting data to and from cloud services incurs bandwidth costs. Hosting an LLM by yourself introduces additional costs and complexities. The initial setup requires a significant investment in hardware, including high-performance servers equipped with GPUs or TPUs. These components are expensive and need regular maintenance and upgrades to stay current with technological advancements. Moreover, the operational costs of self-hosting include electricity, cooling, and physical space for the servers. Running powerful hardware continuously generates substantial heat, necessitating efficient cooling systems to prevent overheating and ensure optimal performance. These operational expenses can add up quickly, making self-hosting a considerable financial commitment [22].

Tokens are the building blocks of text used in NLP and LLMs tasks. They can be as short as one character or as long as one word, depending on the language and context. A token can represent various elements in a text. It might be a word or part of a word in English, a punctuation mark, or even a special character. For example, the sentence "I love AI!" can consist of four tokens: "I", "love", "AI", and "!". The tokenization process splits text into these smaller units, which the LLM then processes individually [23].

LLMs like those developed by OpenAI often have a limit on the number of tokens they can process in a single input. Exceeding this limit can result in truncated responses or errors. Additionally, the number of tokens directly impacts the computational cost and response time, as more tokens require more processing power. To accurately count tokens, one can use specific tools or functions provided by the LLM's API. These tools consider various factors, such as language-specific nuances and encoding schemes. For instance, in OpenAI's models, spaces and punctuation are treated as separate tokens, and certain languages might combine characters differently, affecting the total count [23].

A context window defines the span of text that a model can consider at one time when

generating or understanding language. The context window is essentially the number of tokens that the model can process simultaneously. For example, if a model has a context window of 512 tokens, it can consider up to 512 tokens of text at once. This span includes words, punctuation, and special characters, which the model uses to understand and generate coherent text. A larger context window allows the model to understand and generate more complex and contextually rich responses, making it particularly useful for tasks like long-form content generation, in-depth question answering, and comprehensive dialogue systems. Conversely, a smaller context window might limit the model’s ability to maintain context over long passages, potentially leading to less coherent outputs. Larger context windows require more computational power and memory, which can increase the cost and time required for processing [24].

The prompt(input) and completion(output) cost per million tokens for each LLM from Openrouter is shown in Table 2.2

Table 2.2: Prompt and Completion Costs for Different Models (per M token) in Dollar

Model	Prompt Cost (per M token)	Completion Cost (per M token)
gpt-3.5-turbo-0125	0.5	1.5
llama-3-8b-instruct	0.07	0.07
llama-3-70b-instruct	0.59	0.79
mixtral-8x22b	0.65	0.65
gpt-4o	5	15
gpt-4-turbo	10	30

2.3 Retrieval Augmented Generation

Retrieval Augmented Generation (RAG) is an innovative architectural approach revolutionizing the effectiveness of LLMs. By harnessing custom data, RAG enhances the capabilities of LLMs, enabling them to deliver more accurate, up-to-date, and domain-specific responses. RAG achieves this by retrieving relevant data or documents relevant to a given question or task and utilizing them as context for the LLM. This integration of real-time external data sources ensures that responses are not solely reliant on static training data, thus mitigating the risk of inaccuracies or outdated information. RAG has emerged as an industry standard, offering organizations a straightforward and cost-effective way to augment LLM models without the need for extensive fine-tuning or pretraining. With RAG architecture, companies can deploy any LLM model and seamlessly integrate it with their own data, thereby facilitating efficient and contextually relevant responses tailored to their specific needs [25, 26].

2.4 OpenRouter

OpenRouter is a platform designed to simplify the integration and use of various Large Language Models (LLMs), including both open-source and closed-source models. It provides a unified interface that allows users to search for, compare, and access different models based on their specific needs. This removes the complexities associated with deploying these large AI models by oneself.

One of the standout features of OpenRouter is its API, which includes a built-in fallback mechanism. This means that if one endpoint fails to respond, the system automatically

reroutes to a backup endpoint, ensuring continuous operation without manual intervention. This is particularly beneficial for developers and businesses looking to implement robust LLM applications without worrying about potential downtime caused by server or API issues. Furthermore, OpenRouter keeps users informed about the latest and most effective LLMs available. As new models frequently enter the market, OpenRouter evaluates these based on actual usage and community feedback, making it easier for users to decide which models might be best suited for their projects. This community-driven approach helps filter out the noise and focus on LLMs that offer genuine value in practical scenarios.

OpenRouter stands out as a valuable tool for anyone looking to leverage the capabilities of LLMs efficiently, ensuring they can focus on creating value with AI rather than managing its complexities [27].

2.5 PDF Parsing Methods

This section outlines the methods used for parsing PDFs. It covers Rule-Based Methods, Deep Learning-Based Methods, and Multimodal approaches. Each method's unique characteristics and applications are described to provide a comprehensive understanding of their advantages and limitations in different scenarios.

2.5.1 Rule-Based Method

Rule-based PDF parsing, as implemented by example PyPDF [28], involves converting PDF content into a sequential string of characters without starting to recognize the document's structure. This method segments the text using predefined rules, such as splitting based on newline characters. Despite its efficiency for simple documents, Rule-based faces limitations with complex layouts. It can struggle to accurately distinguish the boundaries of paragraphs and tables, often incorrectly merging or splitting them. The method can also fail to maintain the structural integrity of tables, where text wrapping in cells can lead to misaligned content. Moreover, rule-based methods do not recognize the logical reading order of text, processing content based on its storage order, which can misplace elements like headers or footers and disrupt the intended flow of information. This approach is best suited for documents with predictable formats, where the simplicity of the rule-based method can handle text extraction without the need for more computationally intensive techniques [8].

2.5.2 Deep Learning-Based Method

Deep Learning based PDF parser, there exists a multitude of options and some different approaches to the methods. One widely used method to PDF parsing is the Optical Character Recognition (OCR), it is a technology that converts text from various sources, like scanned documents or images, into editable and searchable digital text. The process involves steps such as image preprocessing, text detection, character segmentation, feature extraction, character recognition, post-processing, and finally generating output in a digital format. This technology enables tasks such as document digitization and text extraction, making it easier to work with printed or handwritten text in digital environments. One model used for PDF parsing is Nougat which utilizes the Tesseract OCR Engine [29, 30].

2.5.3 Multimodal

Multimodal PDF parsers like the Multi-Modal Automated Academic Papers Interpretation System (MMAPIS) are advanced tools designed to enhance the accessibility and comprehension of complex PDFs by integrating and processing the diverse types of information in them. These systems start by extracting all relevant data from the PDF, including text, images, tables, and other embedded content. Each type of data is then processed to convert it into a usable format; text may be structured to maintain its original format, while visual information is preserved accurately to keep its context intact. After processing, the data is organized by aligning it according to the document's sections or chapters. This organization ensures that all information relevant to specific topics remains grouped together, preserving the logical flow and the context as in the original document. The final step involves summarizing this information to make it easier to digest and presenting it through various interfaces. These may include textual summaries and interactive Q&A systems. In general, these parsers not only enable a deeper understanding of PDFs by linking the gap between diverse data types, but also can improve the interaction with the content, making complex information more accessible and easier to interpret [31].

2.6 PDF Parsing Tools

This section details various tools used for parsing PDFs which are based on the Methods explained in section 2.5. These tools include PyPDF, PyMuPDF, Unstructured, Pdfminer.six, Llama Parse, and Llama2 aided Tesseract. Each tool's functionalities and strengths are described, highlighting their effectiveness in handling different aspects of PDF parsing such as text extraction, table recognition, images and handling complex document structures.

2.6.1 PyPDF

PyPDF is an open-source Python library designed to work with PDF files. It's a pure-Python library that allows you to split, merge, crop, and transform the pages of PDF files. It can also extract information from PDFs, such as metadata and content, although its capabilities are somewhat basic compared to more specialized tools for text extraction [32].

PyPDF works by first finding and reading 2 components of a PDF file: the cross-reference table and the trailer. The cross-reference table, or xref table, is a lists the locations of the objects inside the file. The trailer lists the root object, often referred to as the Catalog. Once these components are identified, PyPDF parses the actual objects found within the PDF. PDFs are complex files comprising various types of objects, including dictionaries, arrays, streams, and basic data types such as integers and strings. PyPDF parses objects and stores the parsed data in an internal structure. The content of a PDF is typically stored in content streams, which are sequences of PDF operators and operands. These streams are decoded by PyPDF by using filters to extract the data [33].

It's important to note that while PyPDF can parse text from PDFs, its text extraction capabilities might not perform well with PDFs that have complex layouts or non-standard text encoding. For more complex text extraction tasks, other libraries might be more effective [34].

2.6.2 PyMuPDF

PyMuPDF, also known as Fitz, is a Python library that allows users to interact with PDF files and other document formats like XPS, OpenXPS, and txt. It is a Python binding for the MuPDF library, a lightweight, fast, and comprehensive PDF toolkit. PyMuPDF offers a range of functionalities from basic tasks such as opening and reading PDF files to more complex operations like modifying content and extracting data [35].

PyMuPDF, as detailed by Robin Watts's is a software library that can render and parse several formats including PDFs. The core C library has an API that is used for interacting in many ways with the documents. MuPDF also provides other tools that is build on top of the API [36].

2.6.3 Unstructured

Unstructured is an open-source library designed to process unstructured documents. The idea with Unstructured is to streamline and optimize data processing workflow with LLMs. Unstructured is a comprehensive system in which some of the core functions include partitioning, chunking, cleaning, and staging data. Unstructured parses PDFs by providing tools to automatically partition and extract content from documents into structured formats. It offers functions to identify and extract various elements within PDF files, including tables, text, and metadata. For example, it can distinguish and structure tables within PDFs using table recognition models, which can be enabled with specific functions. This process converts the raw data into structured formats that are easier to analyze and use in downstream applications. [37, 38].

2.6.4 Pdfminer.six

PDFMiner.six is a library used for extracting text from PDF documents by interpreting the arrangement of characters within them. PDFMiner.six handles the unstructured nature of PDFs by using a series of algorithms that analyze the spatial relationships between characters to reconstruct the intended text layout. The process starts by grouping characters that are close together into words. This is based on their positions and proximity, using parameters that define acceptable distances for characters to be considered part of the same word or line. After forming words, these are then organized into lines by evaluating their alignment and closeness. Afterwards, lines are grouped into text boxes, assessing which lines are close enough vertically and horizontally to form coherent blocks of text. This helps in identifying paragraphs and distinct sections of text. The final step involves organizing these text boxes into a hierarchical structure that represents the logical flow of the document. This structured output mimics the way text would be organized in more straightforward document formats. PDFMiner.six also handles vertically oriented text, making it versatile for extracting text from documents that include different text orientations. Through these steps, PDFMiner.six converts the visually oriented layout of PDFs into structured text data suitable for further processing [39].

2.6.5 LLama Parse

LlamaParse is a sophisticated parsing tool designed for handling complex documents, including PDFs and text files with tables, figures, and charts. It integrates seamlessly with

LlamaIndex's ingestion and retrieval services, enabling the creation of robust retrieval systems for semi-structured documents. LlamaParse can convert these documents into markdown format, ensuring accurate query responses across both tabular and unstructured data.

Compared to PyPDF, LlamaParse offers enhanced capabilities for processing semi-structured data. While PyPDF focuses on basic PDF manipulation, LlamaParse excels in transforming complex documents into a more usable format for advanced applications like Retrieval-Augmented Generation (RAG). PyPDF is useful for straightforward PDF tasks, but LlamaParse's ability to integrate with advanced retrieval systems and handle a wider variety of document types makes it more powerful for sophisticated document analysis and query tasks. Currently in public preview, LlamaParse plans to support additional document types in the future, further expanding its utility for building advanced RAG applications [40].

2.6.6 Llama2 aided Tesseract

Llama2 aided Tesseract is an open-source tool which utilizes the Tesseract OCR engine and the Llama2 13B model. Llama2 is used to refine the accuracy of the OCR results, facilitating the conversion of scanned PDFs into legible text files. The model also corrects errors and enhances text formatting. Additionally, the project features a filtering mechanism to identify and remove potential inaccuracies from the model-corrected text, improving overall OCR quality.

The workflow initiates by converting PDFs into images using the pdf2image library, followed by OCR processing of each image using pytesseract. Parallel processing is enabled through the multiprocessing library for efficiency. The Llama2 13B Chat model then refines the OCR output, with options available to verify English validity and reformat text using markdown. Furthermore, it has a function to filter potential hallucinations from the LLM corrected text using sentence embeddings and cosine similarity to compare with the original OCR text.

Finally, it is possible to implement this tool in your own way, this enables the use of other models than the Llama2 13B model [41].

3 Method

This chapter explains the research methodology, primarily a controlled experiment preceded by a literature review. The initial phase of the method is a literature review which lays the foundation for the experiment by providing a comprehensive understanding of the research topic. The controlled experiment then investigates the performance of the Parsing Tools/Methods and LLMs on varying complexities of the PDF files. Lastly, the reliability, validity and ethical considerations are addressed.

3.1 Research Project

Our research project aims to investigate the performance of Parsing Tools/Methods when parsing pdf files into an LLM using a controlled experiment approach. The objective is to understand how the Parsing Tools/Methods and LLMs perform on different complexities of pdf files.

To lay the foundation for the controlled experiment, a literature review was conducted using specific keywords related to our research topic. They were chosen due to their relevance to the project’s techniques and goals. The keywords and their search results are listed in table 3.3.

Table 3.3 contains estimated search results from Google Scholar. We chose articles primarily based on their publication year, focusing on those published from 2020 onwards. We made this choice because our research topic has evolved significantly in recent years, and we aimed to include the most recent and up-to-date articles. It exists many relevant articles meeting the criteria, most articles were from 2023-2024.

Table 3.3: Table of keywords and hits

Keywords	$\approx$ Number of hits
PDF parsing	172 000
LLM	38 900
Multimodal LLM	9 350
PDF parsing LLM	8 090
PDF parsing multimodal LLM	2 440
PDF structure Recognition LLM	31 700
Data Extraction PDF LLM	19 000

3.2 Method

The used method is controlled experiment, using knowledge gained from a literature review conducted during the initial phase. The literature review allows for a better understanding of LLMs, Parsing Tools and Methods, it also provides insights for PDF parsing for use with LLMs.

The controlled experiment evaluates various PDF Parsing Tools and LLMs as independent variables, focusing on their impact on dependent variables that is the how well the LLMs can answer the questions and the cost of using the LLM.

The quality of parsing is determined by the LLM’s effectiveness in understanding and interpreting the content of the parsed PDF files, these files are parsed with the Parsing Tools. How well the parsing tools can parse the content of the PDF will influence the ability for the LLM to give a correct answer to the questions asked about the PDF. For each of the PDF there are several questions that are is related to the content of the PDF file. Additionally, the prompt cost is measured for the LLMs when they are prompted with the parsed files. Cost-efficiency is measured as the balance between prompt cost and performance. The PDF Parsing Tools and their corresponding method are listed in Table 3.4. The LLMs used are listed in Table 3.5.

Table 3.4: Table of PDF Parsing Tools and their Methods

Parsing Tools	Parsing Methods
PyPDF	Rule Based
PyMuPDF	Rule Based
Pdfminer.six	Rule Based
Pdfminer	Rule Based
Pdfminer_as_HTML	Rule Based
Unstructured_OCR_only	Deep Learning Based
Unstructured_high_res	Deep Learning Based
textract_raw_ocr	Deep Learning Based
textract_filtered	Deep Learning Based
textract_corrected	Deep Learning Based
Unstructured_default	Multimodal
Llama_index_txt	Multimodal
Llama_index_md	Multimodal

Table 3.5: Table of LLMs

GPT-3.5-Turbo
GPT-4-Turbo
GPT-4o
Llama3 8B
Llama3 70B
Mixtral8x22B

The data consist of five different PDF files, where the content is gathered from Wikipedia. The choice to gather content from wikipedia was based on articles relevant to our research topic were several related works did the same.

- The first file contains only text in different formats to evaluate how well the tools can work with text based files.
- The second file contains only tables. The is a variety of complexity of the tables, ranging from basic with only two columns to more complex ones with several columns, multi line text, and containing images.
- The third contains images of text and tables and other images containing a variety of information from the dataset in the paper by Y. Liu et al in the [42].

- The fourth contains a combination of text, tables and images in different formats and complexity.
- The fifth file contains text, tables and images in a complex combination.

The questions are based on the content of the different files. The answer to the question is based on the information that is included in the file.

The experimental process include the following:

- Implementing and running the Parsing Tool
- Read and interpret the PDFs with the Parsing Tools, save the interpreted PDF content to a file
- Prompt the LLMs using the parsed files and the corresponding questions.
- Save the answers
- Evaluate and analyze the answers from the steps above

The experimental process is repeated for each tool with the five different complex PDFs. The result is summarized and presented through visual representations.

With this test, there will be thousands of results. To save time and reduce bias in this experiment, an LLM was used for grading the results. To evaluate if the LLM was capable of grading effectively, a random selection of questions and answers was chosen and graded manually, then by the LLM. An LLM was used to construct a prompt suitable for grading these types of questions and answers. Finally, the results graded by humans and those graded by the LLM were compared.

3.3 Reliability and Validity

In this subsection, the concerns of reliability and validity in this project is addressed. The expectation for the consistency of the results and the authenticity of the results are outlined.

3.3.1 Reliability

In this project, we anticipate that other individuals will achieve the same outcome when running it on devices with equivalent computational power. Of course it is crucial that the same Parsing Tools/Methods and LLMs are used to ensure the reproduction of the results. However, there is two concerns regarding the reliability, the first one being that the LLMs may provide different answers each time it's used, rather than always giving the same response. This variability introduces uncertainty, meaning outcomes could not be consistent. The second concern is that some of the outputs from the LLMs are in free-text format, which subjects the evaluation process to our interpretation of the results, based on our assessment of the model's accuracy in interpreting PDF content.

3.3.2 Validity

In this project, the results and discussion are based on the chosen Parsing Tools/Methods and LLMs, therefore we can only make claims based on them. There exists very many different LLMs and it is not impossible that some of them, if tested could change the outcome of this project. Keep in mind, that the choice of Parsing Tools and LLMs for this project is based on articles relevant to our research topic which makes them the most suitable for the task. Furthermore, the result and discussion are based on the performance of the Parsing Tools and LLMs, it ensures that the presented result is authentic and have not been manipulated. additionally, the claims made within the result and discussion is supported by tables, graphs and figures.

3.4 Ethical considerations

This project requires no ethical considerations since it involves no personal information. It does not involve any experiments or surveys where we gather personal data, this project focuses on firstly, conducting a literature review and using existing data and resources related to our research topic. Secondly, the data gathered from the literate review is used to conduct a controlled experiment.

3.5 Team Work

Per is responsible for 1.1, 1.3, 1.6, 2.2.2, 2.3.1-2.3.3, 2.4.1-2.4.5, 3.2, 4, 6,
Daniel is responsible for 1.7, 1.2, 1.4, 1.5, 1.8, 2.1, 2.2.1, 2.2.3, 2.2.4, 2.4.6, 3.1, 3.3, 3.4,
5, 7, 8

4 Implementation

This chapter details the implementation process of the project. It includes the specifications of the hardware and software used, the methods employed for parsing PDFs, the prompting of LLMs, the scoring of the answers by AI, and the visualization of the results. Additionally, it covers the approach for grading the answers, including prompt engineering and the evaluation of manual versus AI grading. Finally, it describes the creation of the PDFs used for parsing in the experiments.

4.1 Hardware

This project is runned and implemtated on one machine, see specifications in Table 4.6.

Table 4.6: Specifications for PC-1

Component	Description
CPU	AMD Ryzen 7 4800HS
RAM	16 GB
GPU	Nvidia GeForce GTX 1660Ti
OS	Windows 11 23H2 64-bit

4.2 Software

The project is implemented in the python programming language. The code is divided in too of four main parts. The code and files can be accessed in Appendix A.

1. Parsing the the five PDF files using the different tools
2. With the parsed PDFs prompt the different LLMs with questions
3. Use a LLM to evaluate the answer
4. Visualize the results

4.2.1 Parsing PDFs

The PDF processing component of the project is designed to parse the PDF files and save the parsed content to output files. Each PDF is processed individually, with a dedicated folder created to store all versions of the parsed content. Throughout the parsing process the performance of metrics execution time and CPU usage are recorded using a custom decorator. These metrics are then saved to an Excel spreadsheet . This section of the project is intentionally single-threaded to ensure accurate measurement of each tool's performance.

In the main workflow, previous output files are cleared, and performance metrics are reset before processing begins. Each PDF file in the input folder is processed using various parsing functions, with the results saved in organized output directories. This approach ensures that parsed data is stored systematically, making it easy to inspect and analyze.

For the tools used see table 4.7

4.2.2 LLM prompting

The question-answering component of the project is designed to process the content from various parsed files and generate responses to the predefined questions using different LLMs. Each text file is processed individually, with the corresponding questions. The responses, along with relevant performance metrics, are stored in an Excel spreadsheet for further analysis.

In the main workflow, the LLMs models are iterated over and then each folder containing text files is iterated over, and each file is processed by a specific LLM. The questions for each file are retrieved from the corresponding question file stored in a designated folder. The text content of each file is then read, and a conversation context is established with the system's instructions to provide concise answers. After each question, the following information is collected and added to a Pandas Data frame:

- Model name
- PDF file name
- Parsing tool
- The asked question
- The answer from the LLM
- Prompt tokens
- Completion tokens
- Total tokens
- Cost for the prompt
- Timestamp for the run
- The correct answer to the question

A concurrent processing approach is employed using `ThreadPoolExecutor` to handle multiple files simultaneously, thus speeding up the processing of the files since the majority of the time is spent waiting for the response from the LLMs. For each question, the language model is prompted, and the responses, along with the previous mentioned data are recorded. In cases where the model fails to provide a response, error handling mechanisms are in place to log the errors. One time it will retry, if that fails a error row will be added to the data frame to indicate a error.

4.2.3 AI scoring of the answers

The next step is to evaluate the responses from the LLMs. The responses was graded in a binary scale. See the Answer grading.

4.2.4 Visualising of the results

To visualize the results of the experiments, a Jupyter Notebook was created. The data analysis was performed using the Pandas library, and the results were visualized using Matplotlib.

4.2.5 Libraries

The software libraries used in this project are listed in Table 4.7.

Table 4.7: Software and Libraries	
Software/Library	Version
python	3.12 64-bit
PyPDF	4.2.0
PyMuPDF	1.24.2
Unstructred	0.13.7
Pdfminer.six	20231228
Pdfminer	20191125
Llama2 aided Tesseract	1
Llama index	0.4.3
Pandas	2.2.2
OpenAI	1.25.0
Jypter Notebook	8.13.2
Poppler	24.02.0
Tesseract	5.3.3

4.3 Answer grading

The next step was to evaluate the vast amount of responses from the LLMs. To do this in an efficient way a LLM was used with a specially engineered system prompt for this task. The grading was done using a binary system. If the answer contained the correct information, it received a score of 1 otherwise, it received a score of 0. The grading focused solely on whether the answer contained the correct information in any form, thus meaning the LLM has been able to interpret the data provided. The manner of presentation did not matter. For example, if the correct answer to a question was "10," then both "10" and "ten" were graded as correct. Any additional information provided by the LLM was not taken into consideration. An attempt was made to add a "not mentioned" field to the prompt. However, this did not improve the grading by the LLM, so it was not used in the grading.

4.3.1 Prompt engineering for LLM grading

The basic prompt: "Validate if the provided answer contains the correct information. Reply with 1 if the answer is correct, otherwise reply with 0" was used as the starting point. Thirty random answers were selected and graded by both a human and a LLM using this prompt. Diverse questions were then given to the another LLM, which was prompted to improve the system prompt. These steps were repeated until there were three consecutive runs without diverging grades.

4.3.2 Evaluating manually grading vs LLM grading

To conduct a more extensive evaluation of the effectiveness of the system prompt for grading answers, 10% of all the answers were randomly selected and manually graded by individuals. This test revealed that the LLM and the human graders agreed on the score 98.2% of the time. This level of agreement was deemed sufficient for the project.

4.3.3 The PDFs

The content for the PDF files were gathered from Wikipedia except for the *AidonHaninterface* file. This file is gathered directly from google. The content from Wikipedia was copied into a Word document, where its format and structure were modified to be more complex. Finally, the Word document was saved as a PDF file.

- *Texts* PDF file contains only text in different formats to evaluate how well the tools can work with text based files. This contains text in different formats, columns, text sizes, header and footer
- *Tables* PDF file contains only tables. There is a variety of complexity of the tables, ranging from basic with only two columns to more complex ones with several columns and multi line text.
- *Images* PDF file contains images of text and tables and other images containing a variety of information from the dataset in the paper by Y. Liu et al in the [42].
- *Mixed1* PDF file contains a combination of text, tables and images in different formats and complexity.
- *AidonHaninterface* PDF file contains text, tables and images in a complex combination. This file has a complex layout with a lot of technical information about the RJ12 HAN-port interface.

The PDF files can be found on the github project, see Appendix A.

5 Experimental Setup and Results

This chapter outlines the experimental setup of the study, detailing the hardware specifications, libraries, frameworks, and software configurations used. Additionally, it presents the results obtained from the experiment.

5.1 Experimental Setup

The experimental hardware specifications used are shown in Table 4.6. The Parsing tools used are listed in Table 3.4 and the LLMs in Table 3.5. The environment is setup in the Visual Studio Code IDE and the software specifications are listed in Table 4.7.

In Section 3.2 the overall approach of this experiment is presented. The primary focus is to evaluate which Parsing Tool/Method that is the most efficient for parsing the five different PDF files with varying complexities. The LLMs have an impact of the results, therefore the LLMs ability to interpret the content of the parsed PDF files is evaluated. The LLMs cost, size and parameters is also taken into account. Section 3.2 explains the PDF files and how the experiment was conducted.

5.2 Results

Figure 5.1 shows the average percentage of correct answers on all files based on the questions in the Pdf files. These numbers represents a mean value of all the Parsing Tools and LLMs. As seen in the figure, Images was most difficult to handle for the tools.

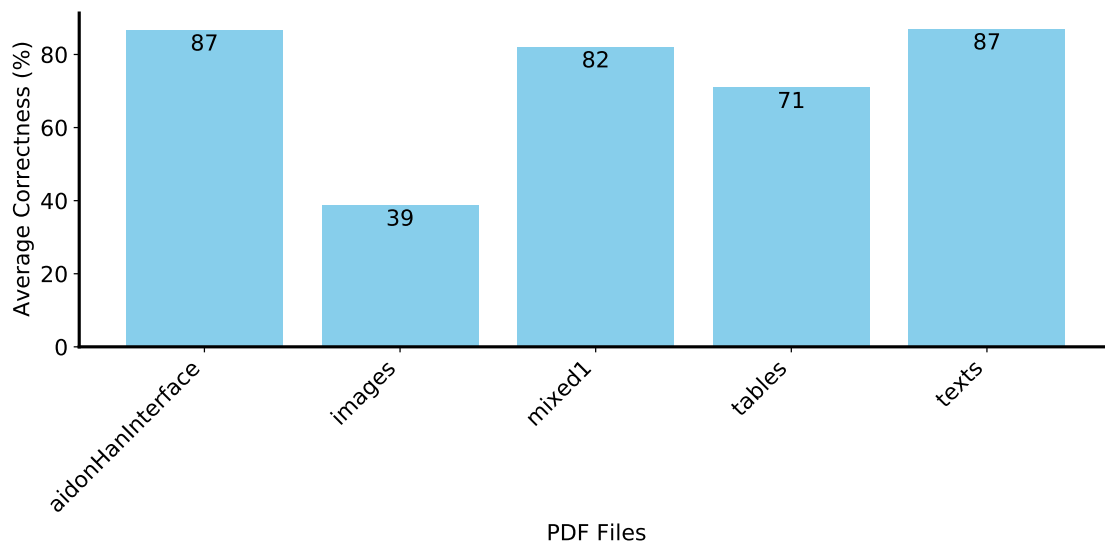


Figure 5.1: Average Correctness per File

Figure 5.2 shows the average percentage of correct answers per LLM based on the questions to all the PDF files. The value represents the average number of correct answers with all the Parsing Tools. GPT-4-Turbo and GPT-4o performed better than the other LLMs.

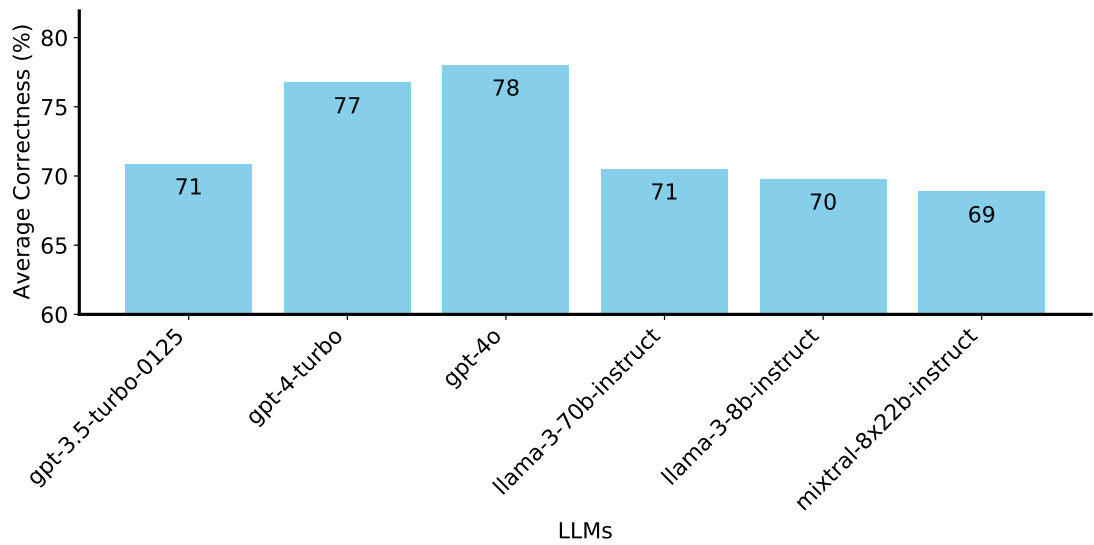


Figure 5.2: Average Correctness per LLM

In Figure 5.3 the average percentage of correct answers per Parsing Tool is presented, this is based on the questions to all PDF files which were corrected with all the LLMs. The multimodal llama_index_md and llama_index_txt performed best with scores of 82% and 83%.

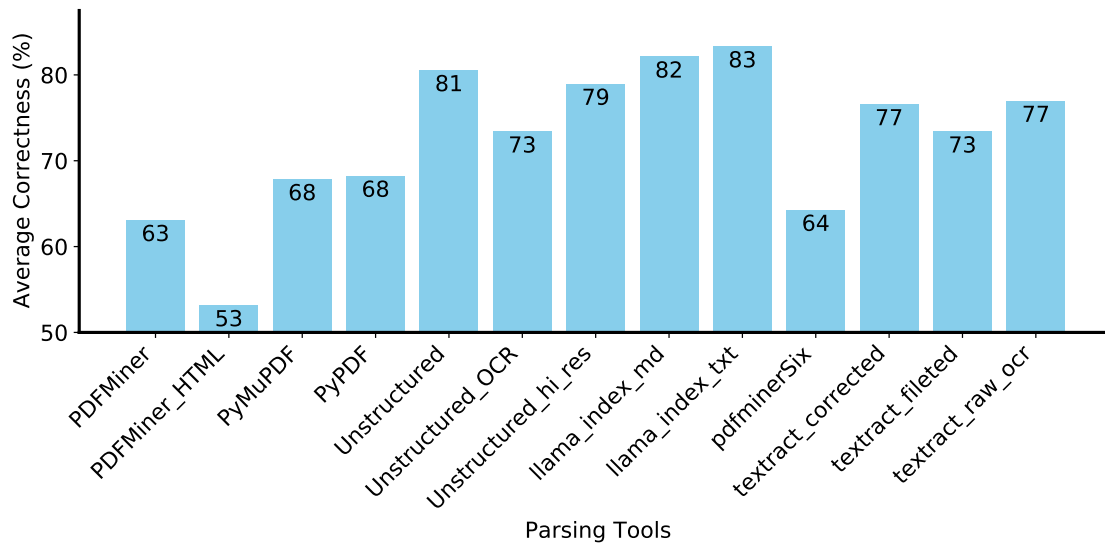


Figure 5.3: Average Correctness per Parsing Tool

Figure 5.4 demonstrates the same thing as Figure 5.3 expect here the Images PDF file is excluded. This showcases how the Parsing Tools performs on PDF files that does not contain complex images and as seen all tools performed significantly better when the images file was excluded. With a score of 87% the Rule Based PyPDF is not far behind the multimodal llama_index_md with a score of 88%.

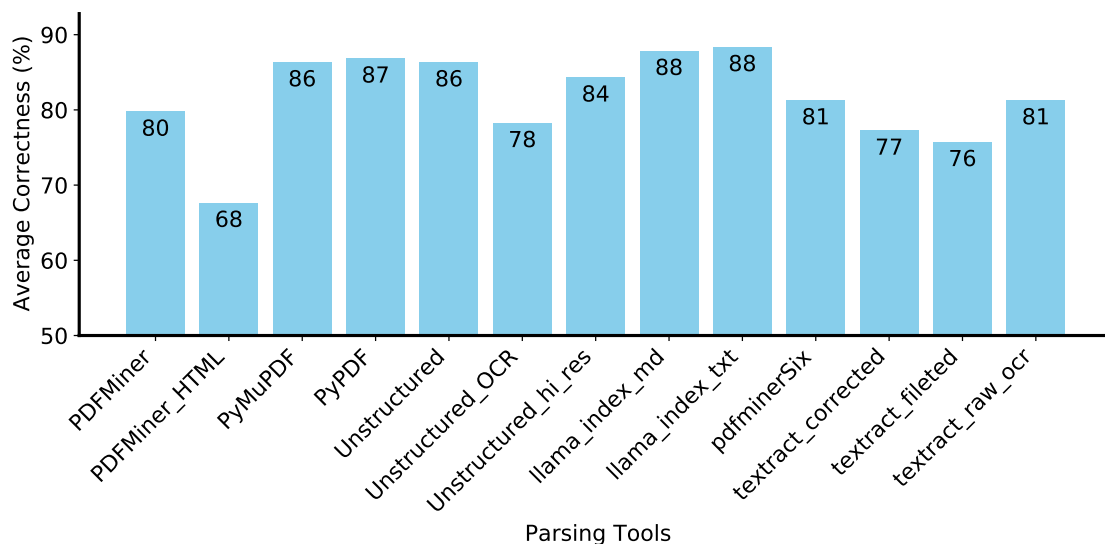


Figure 5.4: Average Correctness per Parsing Tool (Excluding Images File)

In Figure 5.5 the average percentage of correct answers for each LLM on each PDF file is shown. The LLMs average value is based on all the Parsing Tools. As seen GPT-4-Turbo and GPT-4o was among the top performing LLMs on each file.

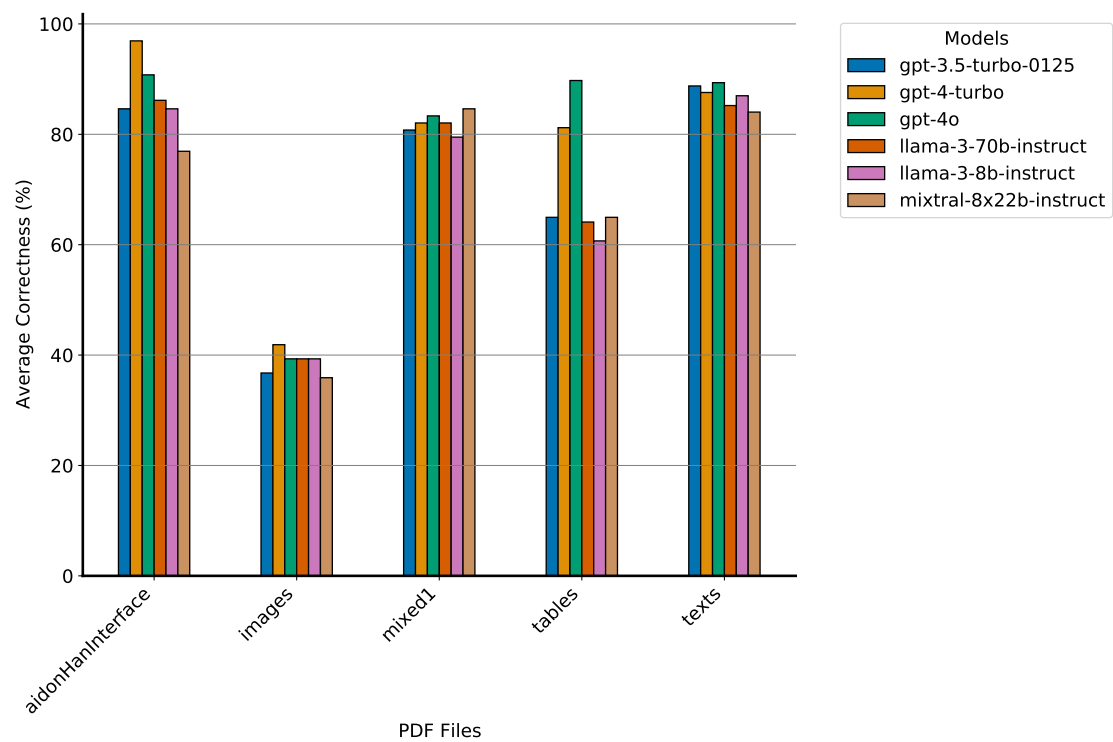


Figure 5.5: Average Correctness per LLM and File

Figure 5.6 showcases the average percentage of correct answers for each Parsing Tool with each LLM. This is based on all the PDF files. The Rule Based PDFMiner_HTML only had a score over 60% when combined with GPT-4-Turbo and GPT-4o, the highest score which was over 90% were the combination of llama_index_txt and GPT-4o.

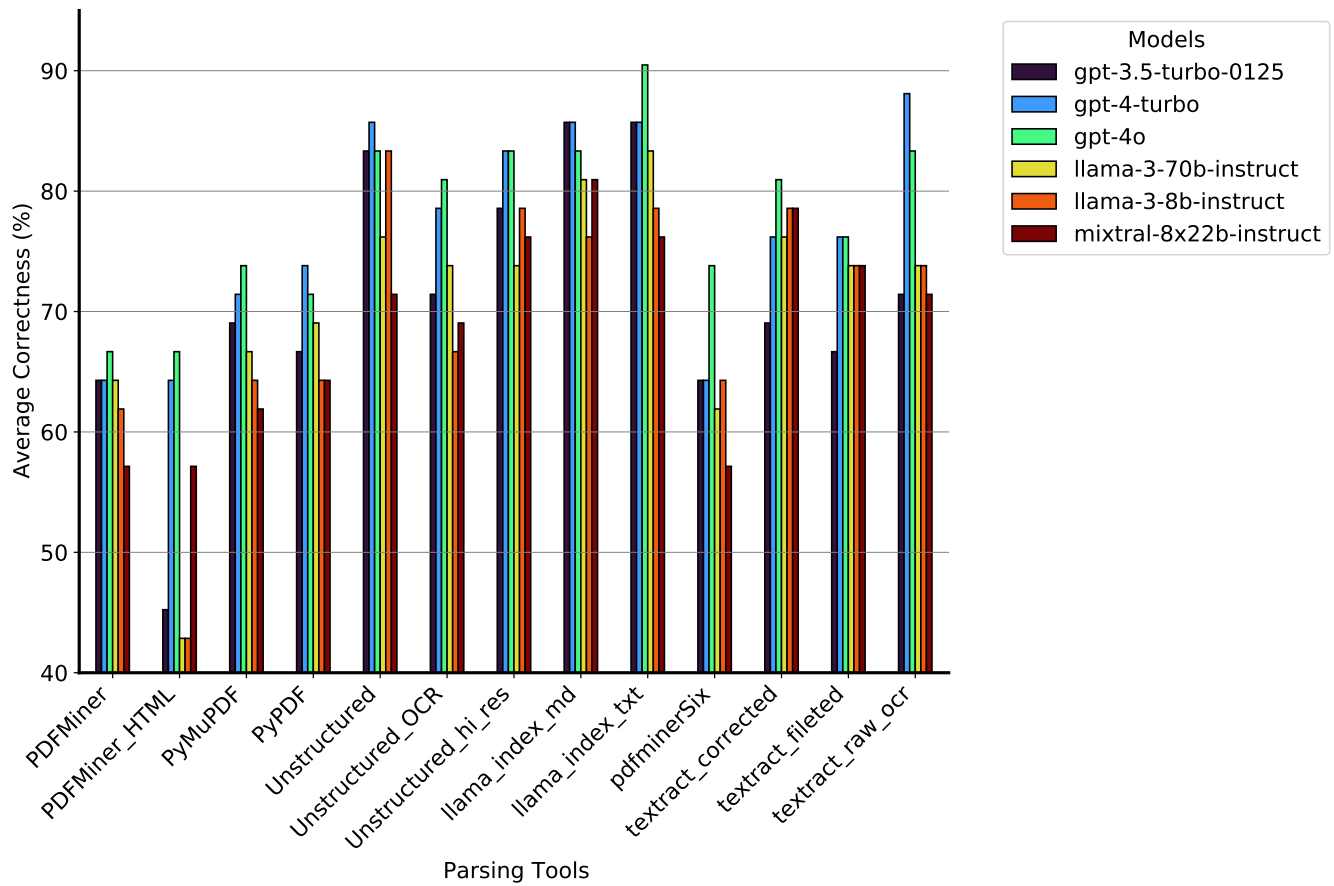


Figure 5.6: Average Correctness per LLM and Parsing Tool

The average percentage of correct answers for each PDF file with each LLM is presented in Figure 5.7. The value represents the average value for the LLM with all parsing tools, sorted from the best performing LLM to the left and the worst to the right. As seen GPT-4-Turbo was superior on the AidonHanInterface file with a score of 89% and mixtral-8x22B were significantly worse with a score of 77%.

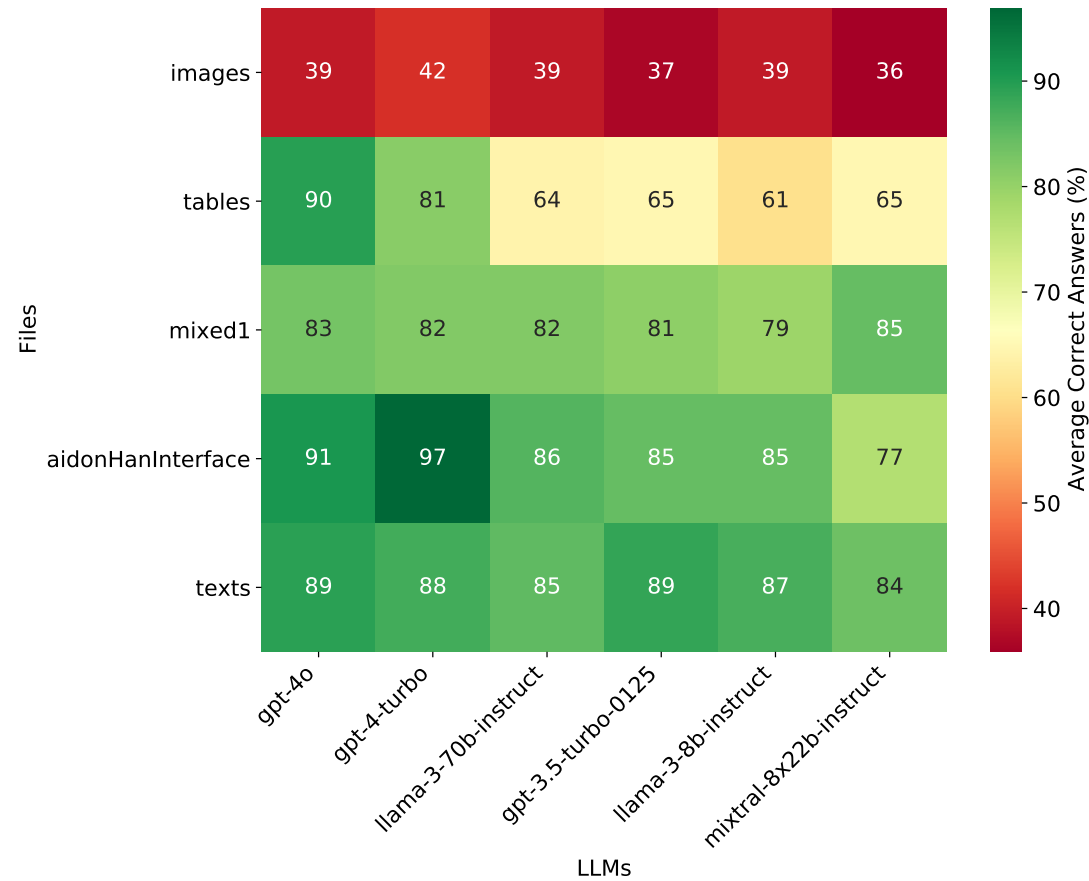


Figure 5.7: Average Correctness per LLM and File (%)

The average percentage of correct answers for each Parsing Tool with all LLMs on each PDF file is demonstrated in Figure 5.8. It is sorted from the best average performing Parsing tool in the left to the worst on the right.

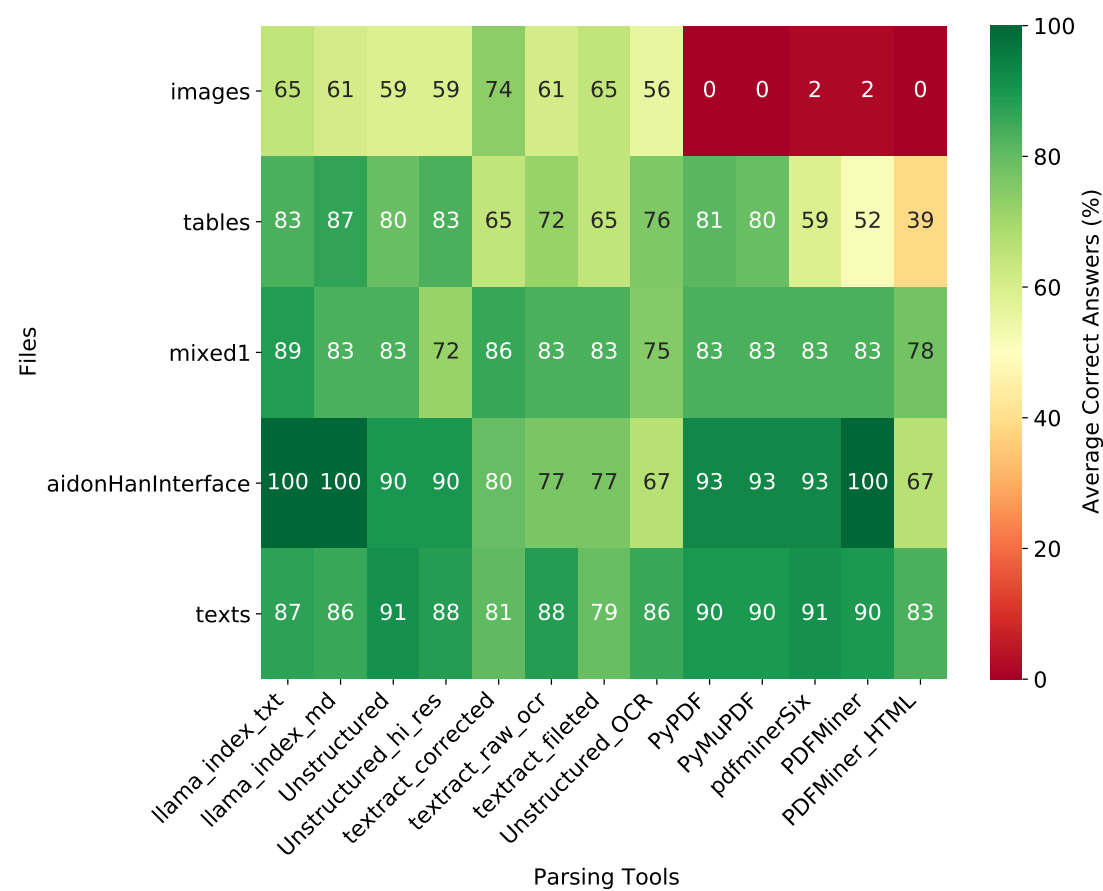


Figure 5.8: Average Correctness per Parsing Tool and File (%)

In Figure 5.9 the average percentage of correct answers for each Parsing Tool with each LLM on all PDF files. The heatmap is sorted from the best performing Tool and LLM on the left and the worst on the right. The multimodal llama_index_md and llama_index_txt is the top two overall performing tools in combination with all LLMs. GPT-4-Turbo and GPT-4o is the top two overall performing LLMs on all tools.

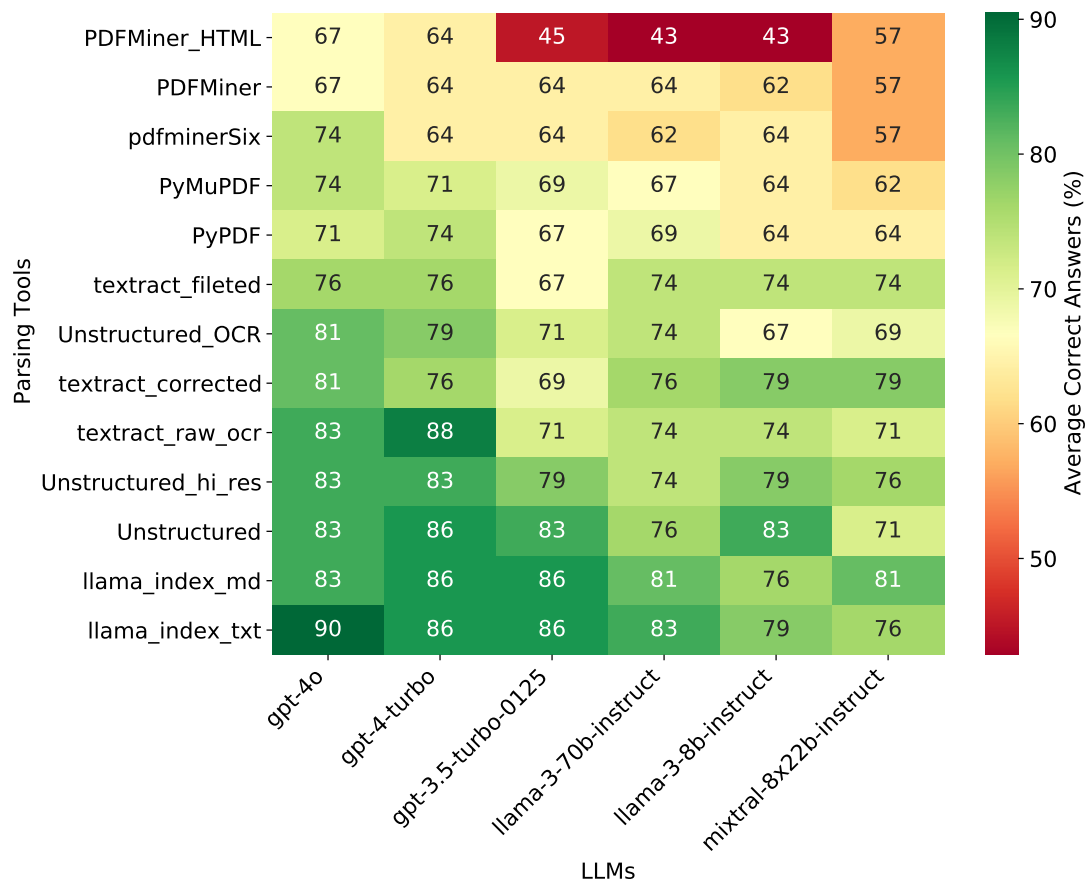


Figure 5.9: Average Correctness per Parsing Tool and LLM (%)

Figure 5.10 present the percentage of correct answers for each Parsing tool with each LLM on only the Aidonhaninterfernce PDF File. The heatmap is sorted from the best performing Tool and LLM in the bottom left corner and the worst in the upper right corner. As seen the performance among most of the tools were equal with scores of 100%.

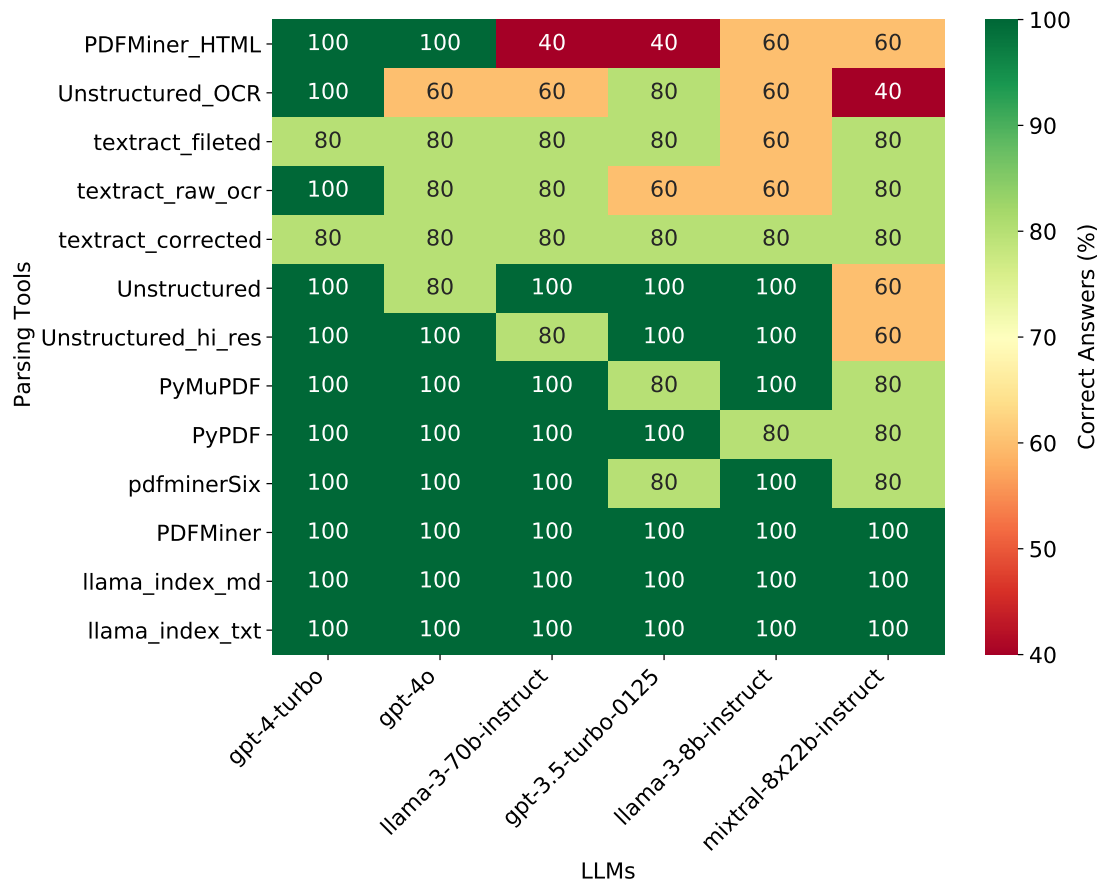


Figure 5.10: Percentage Correct Answers for Aidonhaninterfernce PDF File

The percentage of correct answers for each Parsing tool with each LLM on only the Images PDF File is presented in Figure 5.11. The heatmap is sorted from the best performing Tool and LLM in the bottom left corner and the worst in the upper right corner. As seen textract_corrected outperformed the other tools with almost all LLMs and the Rule Based tools could not handle the Images at all(Seen with the red color).

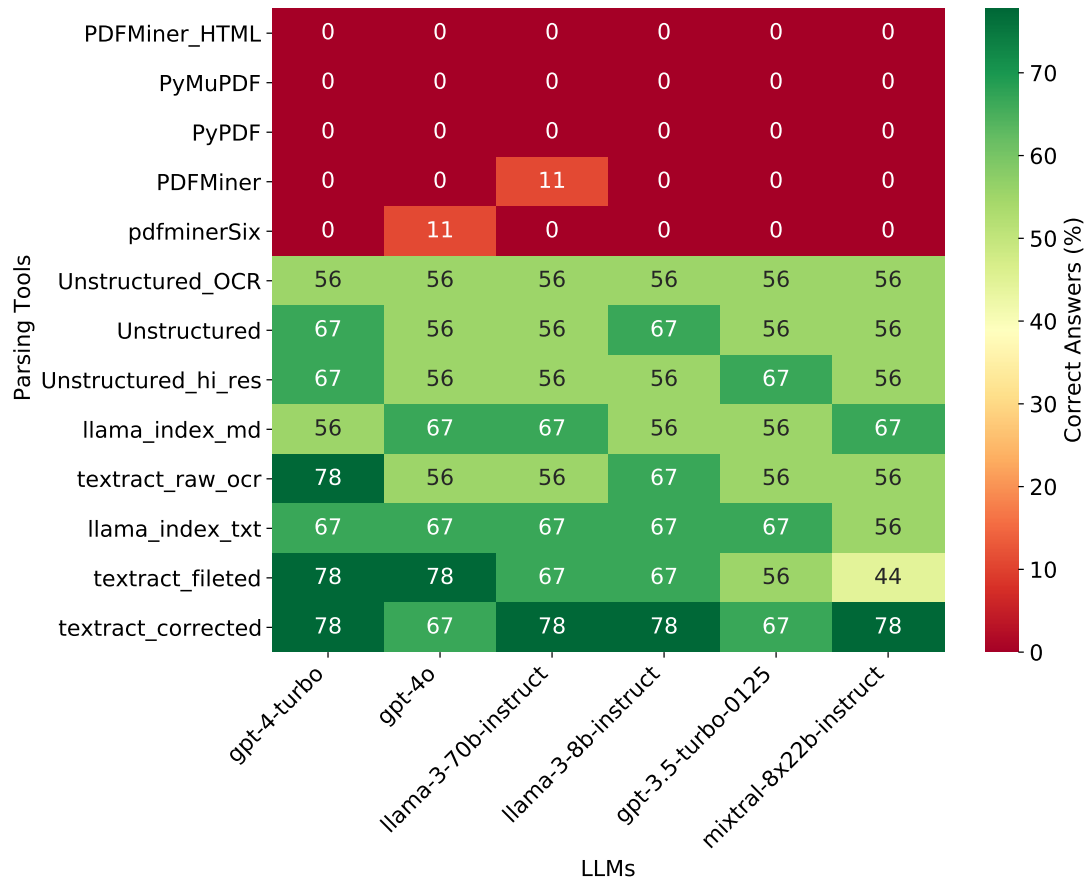


Figure 5.11: Percentage Correct Answers for Images PDF File

Figure 5.12 present the percentage of correct answers for each Parsing tool with each LLM on only the Mixed1 PDF File. The heatmap is sorted from the best performing Tool and LLM in the bottom left corner and the worst in the upper right corner. The performance among the tools in combination with the LLMs were equal, the score of 100% with textract_corrected and mixtral-8x22B-instruct were discovered as an error in the AI correction.

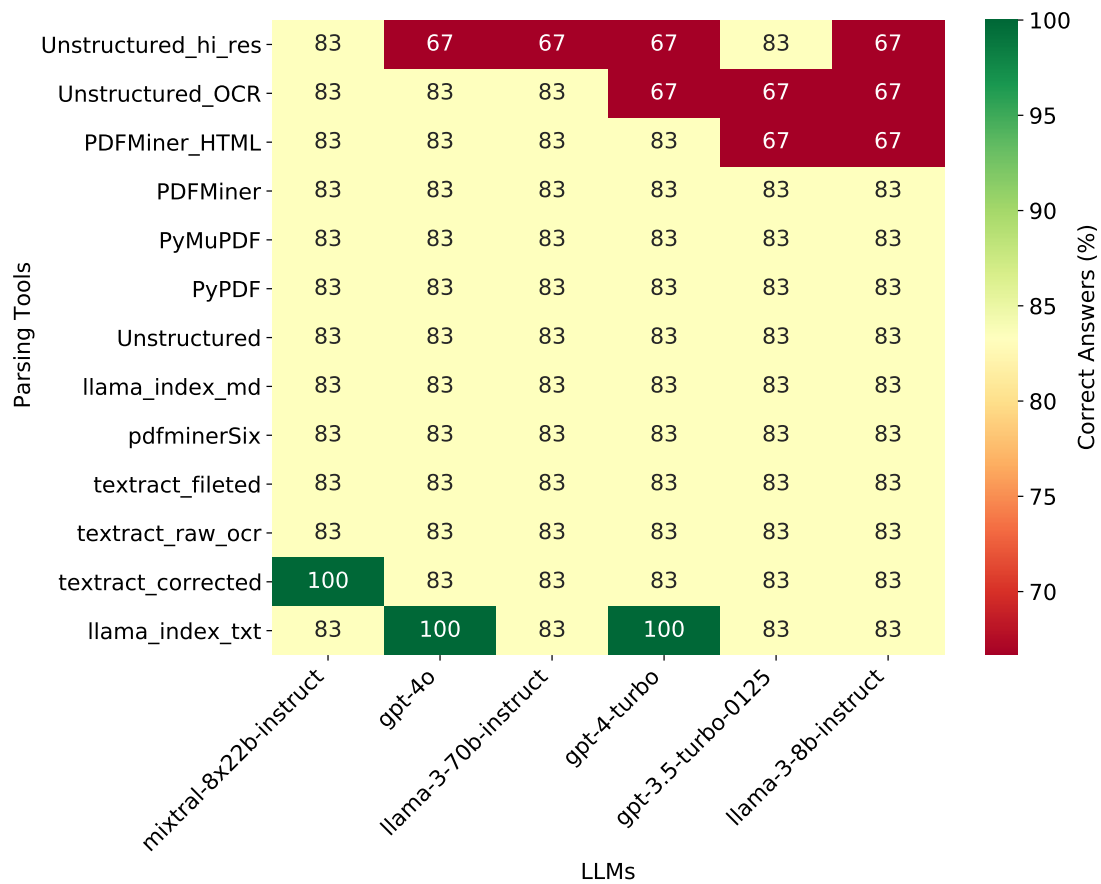


Figure 5.12: Percentage Correct Answers for Mixed1 PDF File

In Figure 5.13 the percentage of correct answers for each Parsing tool with each LLM on only the Tables PDF File is presented. The heatmap is sorted from the best performing Tool and LLM in the bottom left corner and the worst in the upper right corner. For the Tables file GPT-4-Turbo and GPT-4o were superior in interpreting the parsed files.

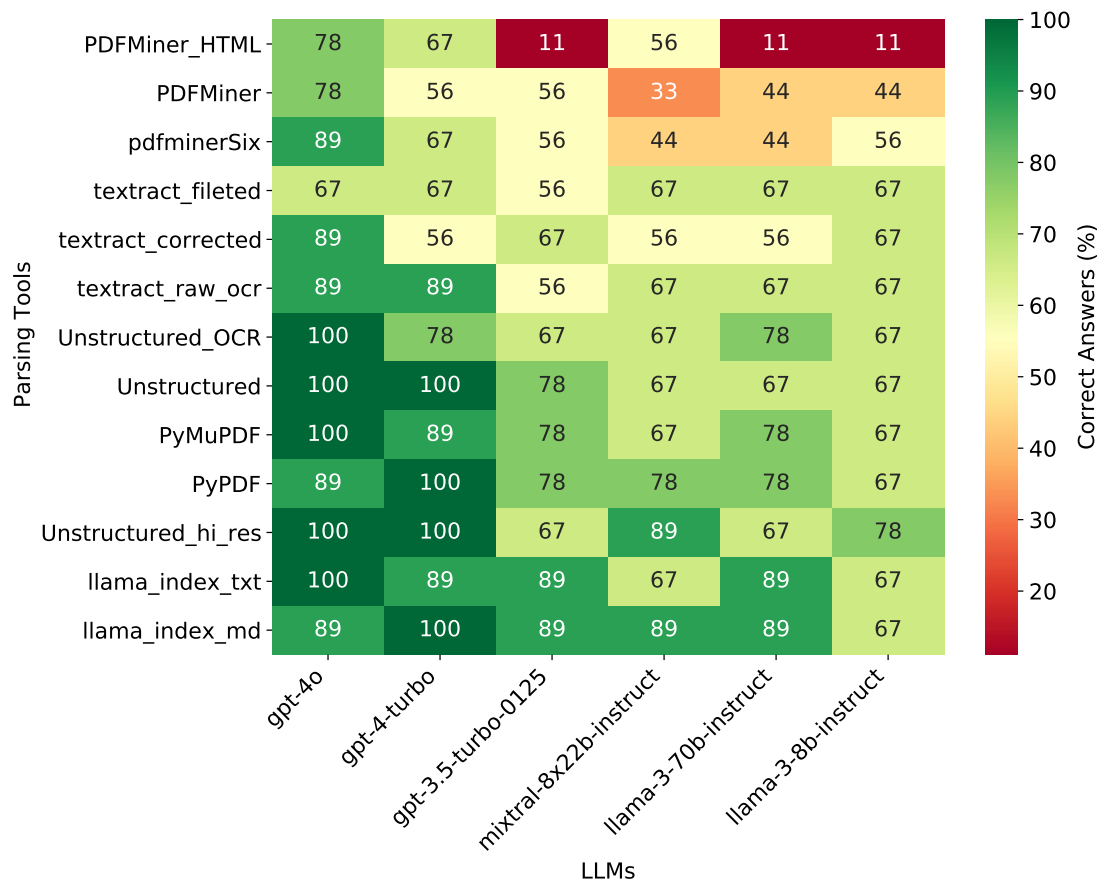


Figure 5.13: Percentage Correct Answers for Tables PDF File

As displayed in Figure 5.14, the percentage of correct answers achieved by each Parsing tool when paired with each LLM is shown, specifically for the Texts PDF File. The heatmap is sorted from the best performing Tool and LLM in the bottom left corner and the worst in the upper right corner. Among the top five performing tools, four of them were Rule Based and one were multimodal. This shows that on the Texts PDF the Rule Based tools performs equal to the multimodal tools.

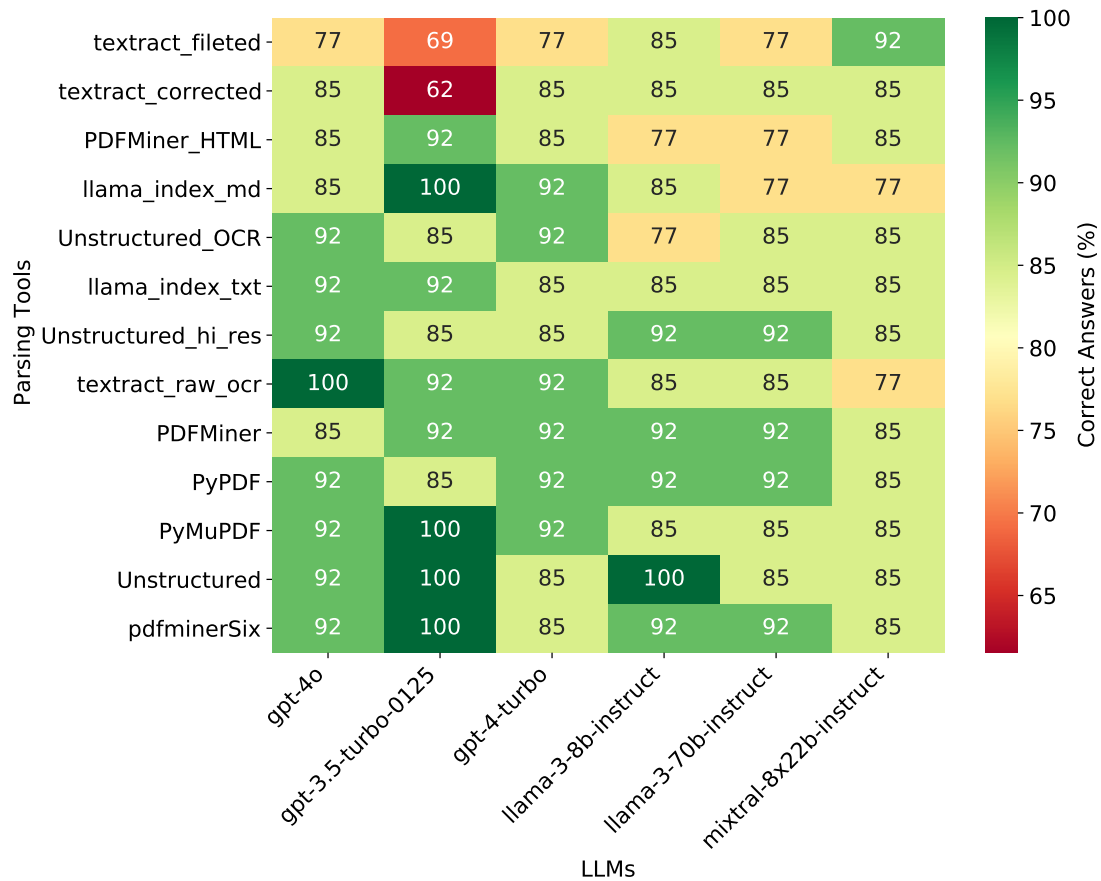


Figure 5.14: Percentage Correct Answers for Texts PDF File

In Figure 5.15 the total cost for prompting the LLMs with the parsed files from each Parsing Tools is shown. This is based on all the LLMs and all PDF files. As seen there is no major differences between the tools expect for PDFMiner_HTML which had significantly higher cost of 8.99 Dollar.

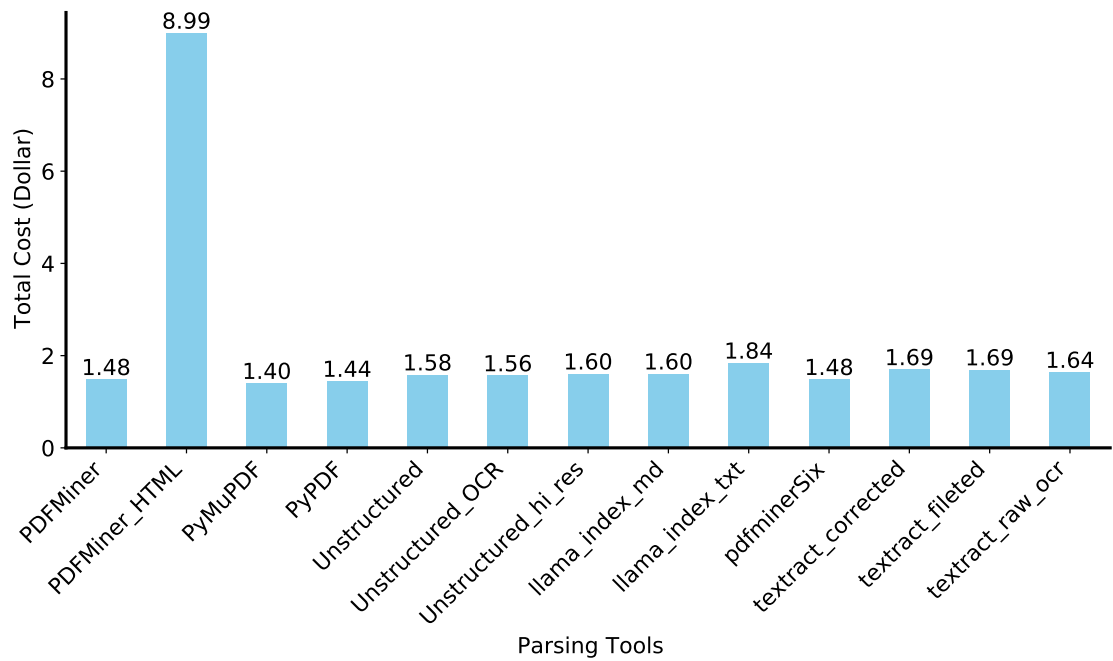


Figure 5.15: Total Prompt Cost with Parsing Tools

Figure 5.16 presents the same thing as Figure 5.15 except here the PDFMiner_HTML is excluded because of its higher cost. This figure shows that the largest difference in cost is 0.44 Dollar, which is between llama_index_txt and PyMuPDF.

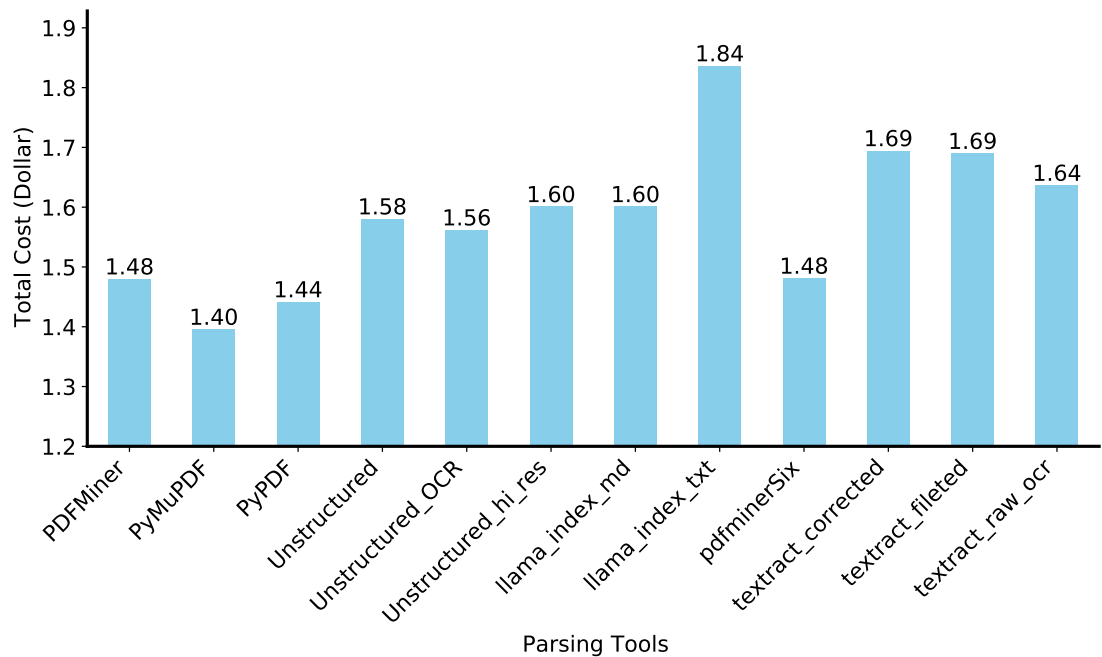


Figure 5.16: Total Prompt Cost with Parsing Tools (Exluding PDFMiner_HTML)

In Figure 5.17 the total cost in US Dollar of prompting the LLMs are shown. As seen there is a significant difference in the cost between the cheapest llama-3-8B-instruct and the most expensive GPT-4-Turbo.

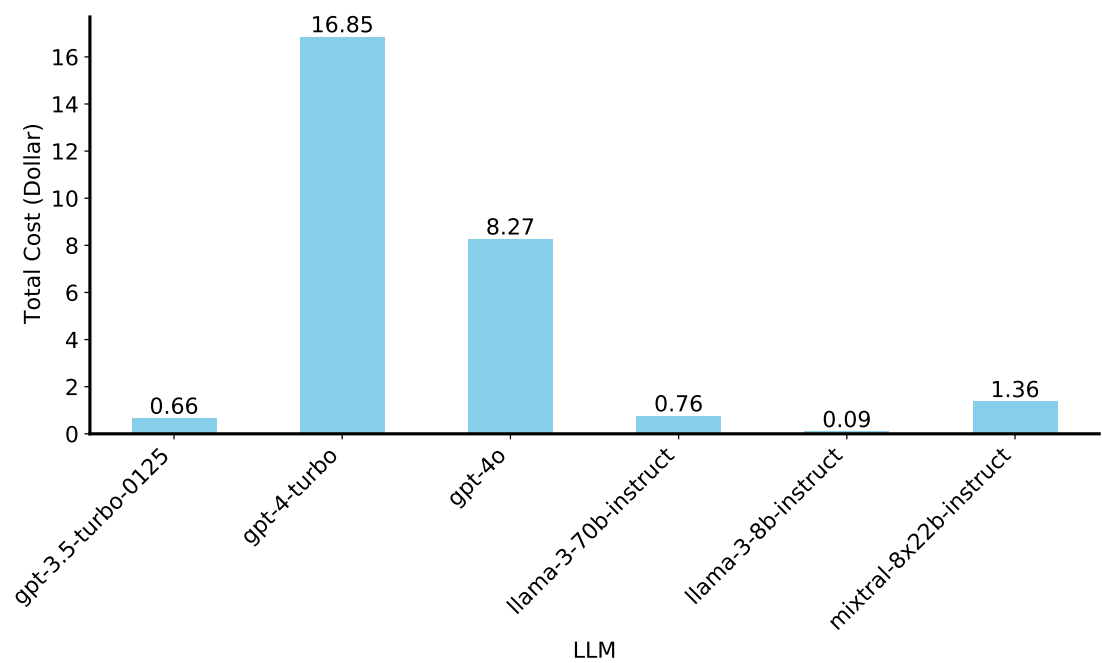


Figure 5.17: Total Prompt Cost on LLMs

As displayed in Figure 5.18, the prompt cost in US dollars for each Parsing Tool when paired with each LLM is shown. The difference in prompt cost depends more on the choice of LLM rather than the choice of parsing tool.



Figure 5.18: Prompt Cost per Parsing Tool and LLM

Figure 5.19 presents the same thing as Figure 5.18 except here is the PDFMiner_HTML Parsing Tool excluded because of its significantly higher cost, therefore the figure presents the difference in prompt cost between the other tools in clearer way.

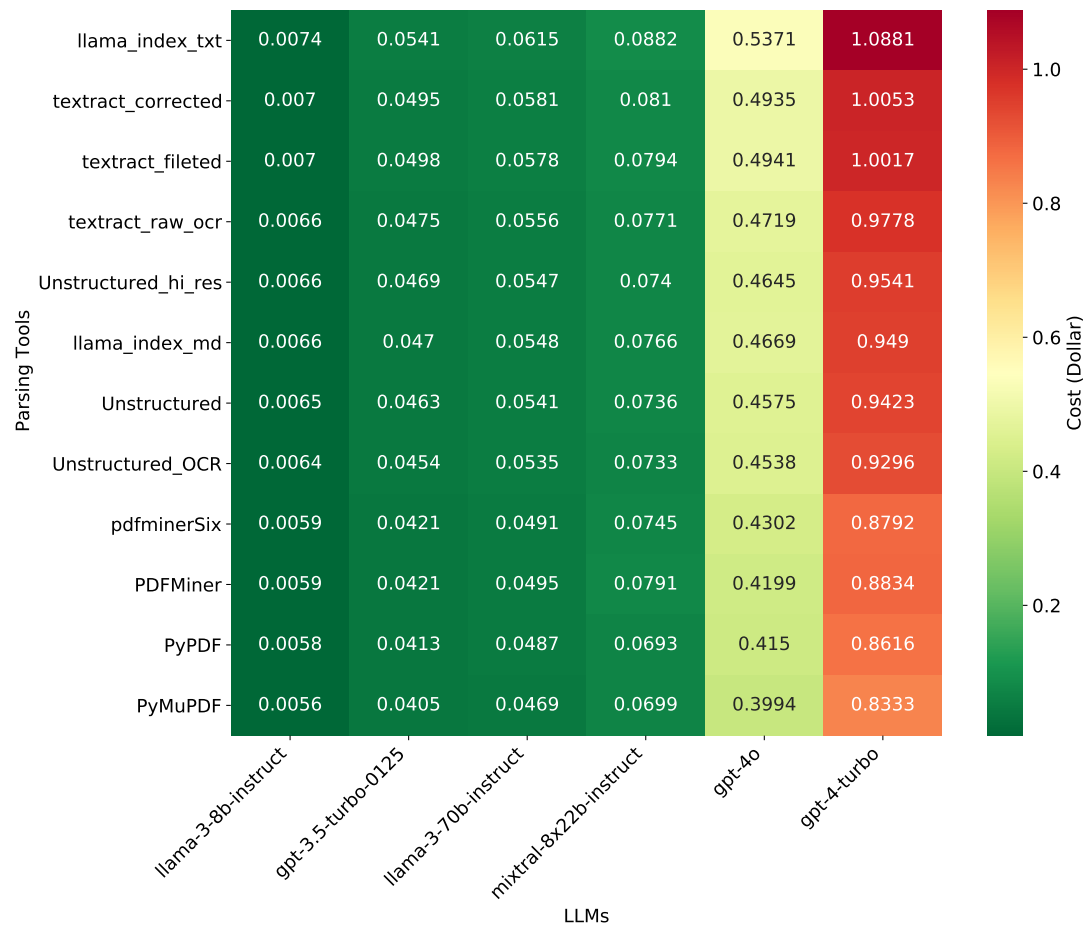


Figure 5.19: Prompt Cost per Parsing Tool and LLM (Exluding PDFMiner_HTML)

As displayed in Figure 5.20, the prompt cost for each LLM with the Parsing Tools on the Aidon-Haninterface PDF file is shown, excluding PDFMiner_HTML.

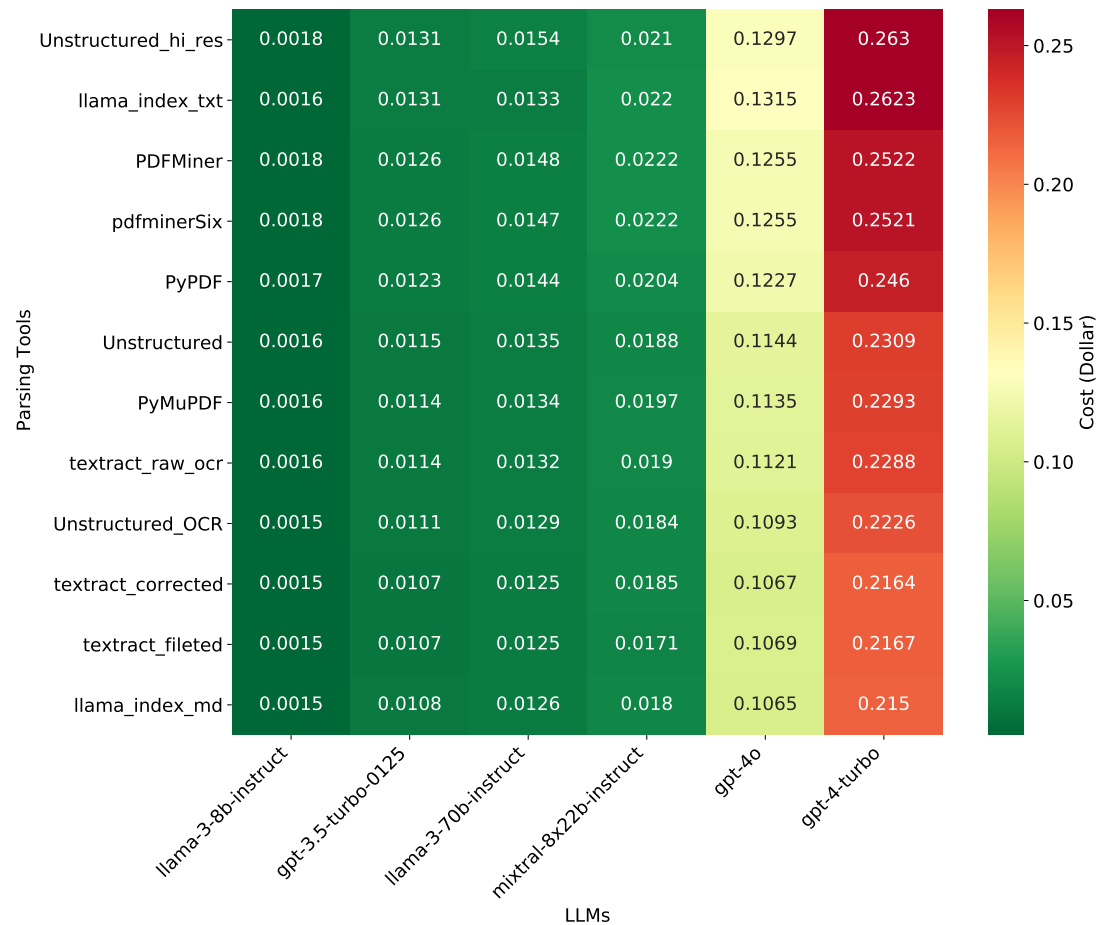


Figure 5.20: Prompt Cost per Parsing Tools and LLMs on the AidonHaninterface PDF file

In Figure 5.21 the Prompt Cost of each LLM with the Parsing Tools on the Images PDF file is presented. The Rule Based PDFMiner_HTML, PDFMiner, PyPDF, PyMuPDF and PDFMinerSix are excluded because they cant handle Images at all and therefore the prompt cost for these are not interesting.

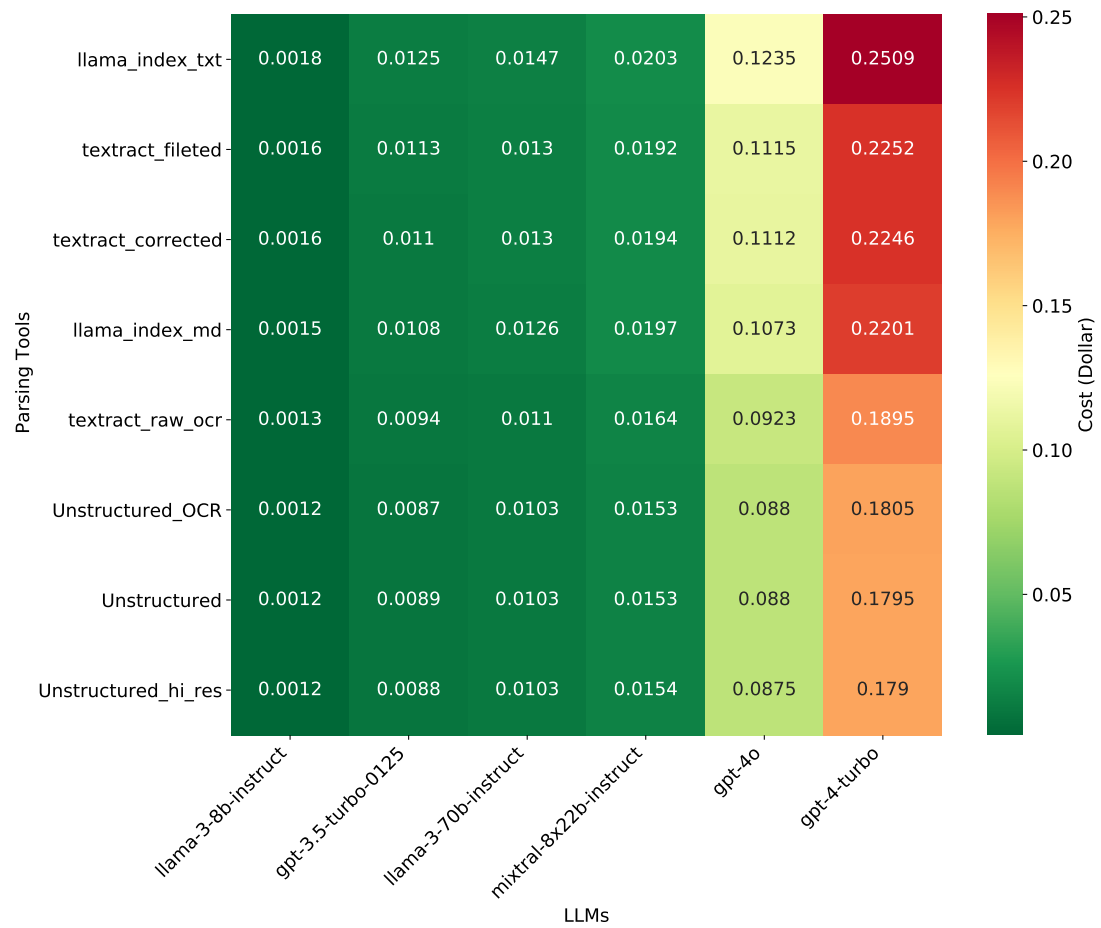


Figure 5.21: Prompt Cost per Parsing Tools and LLMs on the Images PDF file

Figure 5.22 presents the Prompt Cost on each LLM with the Parsing Tools on the Mixed1 PDF file. The PDFMiner_HTML is excluded.

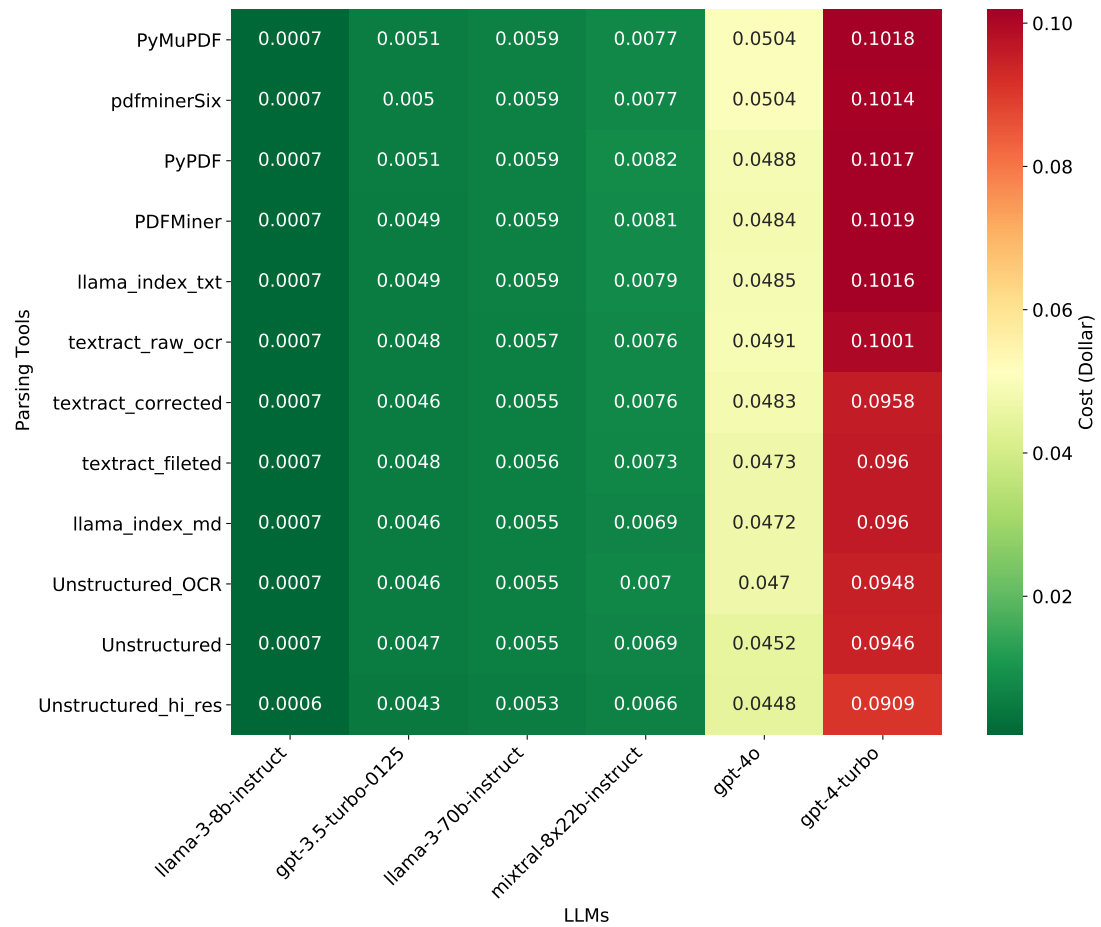


Figure 5.22: Prompt Cost per Parsing Tools and LLMs on the Mixed1 PDF file

In Figure 5.23 the Prompt Cost on each LLM with the Parsing Tools on the Tables PDF file is preseted, where PDFMiner_HTML is exluded.

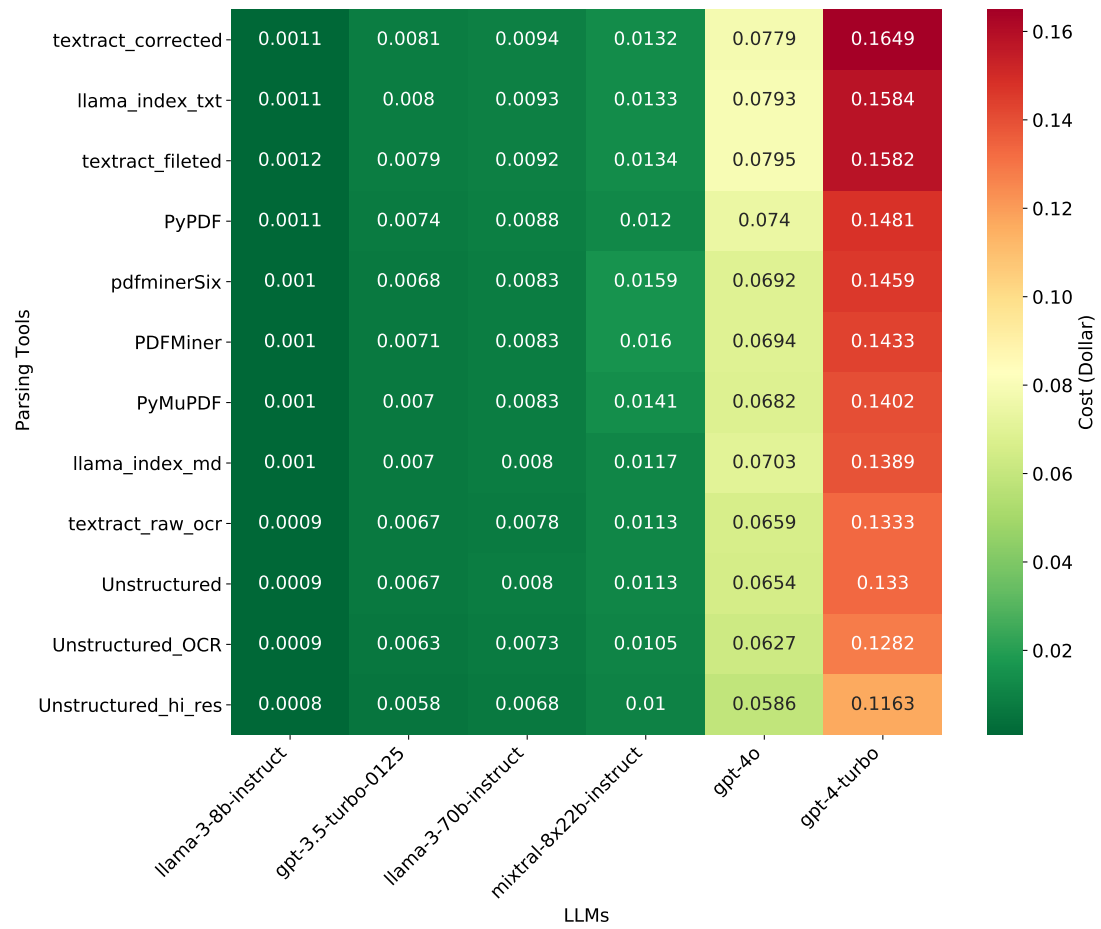


Figure 5.23: Prompt Cost per Parsing Tools and LLMs on the Tables PDF file

Figure 5.24 shows the prompt cost for each LLM paired with the Parsing Tools on the Texts PDF file, excluding PDFMiner_HTML

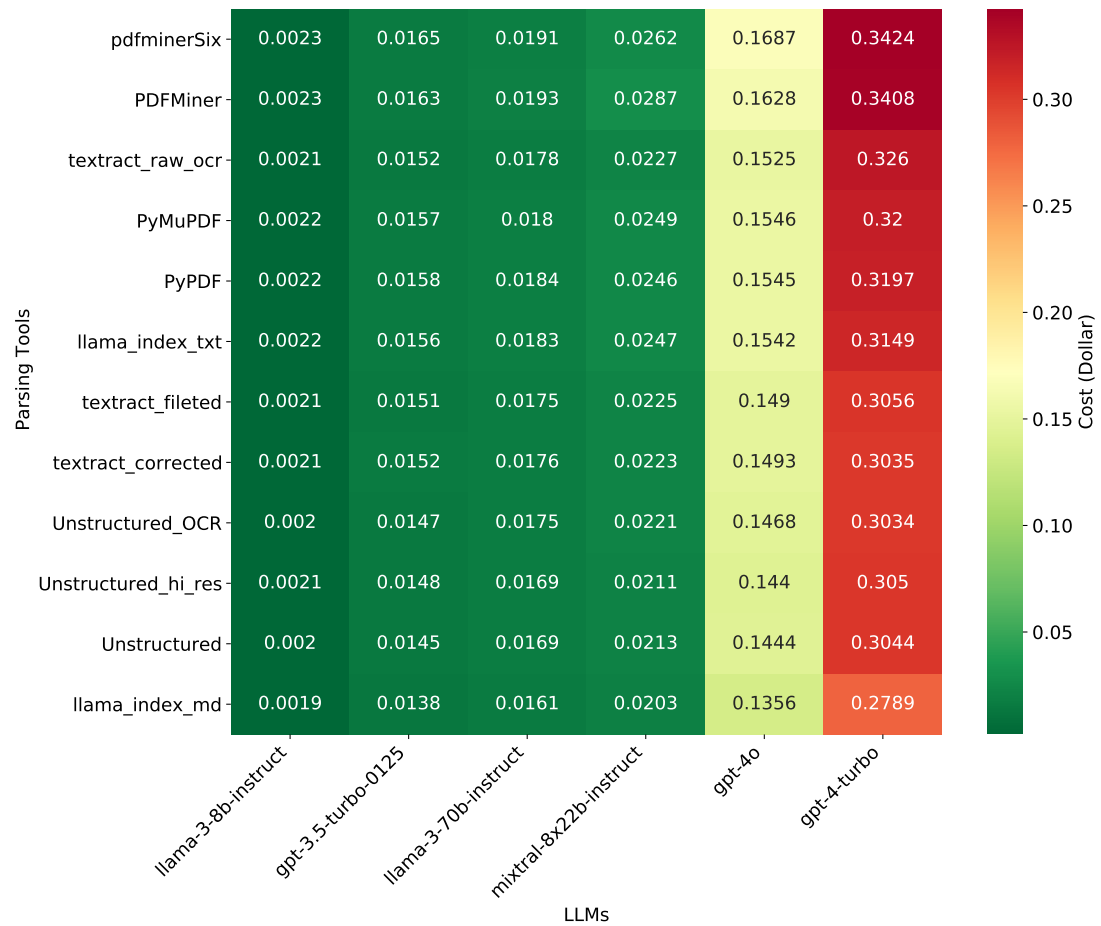


Figure 5.24: Prompt Cost per Parsing Tools and LLMs on the Texts PDF file

6 Analysis

There is a significant difference of the performance on the PDF files as shown in Figure 5.1, with the score of only 39% on the Images file compared to the others with minimum 71% in average correctness. The low score on the images file is because the Rule Based Parsing Tools do not have the capability to handle Images. The Parsing Tools managed to interpret the Texts and aidonHaninterface PDF files equally good with a score of 87% on both files. The Tools had a score of 82% on the Mixed1 PDF file and a score of 71% on the Tables file. The results demonstrate that PDF files with complex images and tables are more difficult for the Parsing Tools to interpret accurately compared to text-based files or those with less complex mixed content. This analysis will take a deeper look into how the specific parsing tools/methods and LLMs perform on the different PDF files.

6.1 Overall performance of the LLMs

Overall, there were no huge differences between the LLMs average performances, they had a score between 69% and 78% as seen in Figure 5.2. GPT-4-Turbo and GPT-4o were slightly better than the other LLMs with a score of 77% and 78%. In theory, GPT-4o performs slightly better than GPT-4-Turbo and as shown in the result there is a small distinction of 1% in their performance, which supports the theory. The other four LLMs had scores within $\pm 1\%$ of 70%, this presents an equal performance between them. The distinction between llama3-8B and llama3-70B is 1%, in theory the llama3-70B model should perform better than the smaller 8B model but as shown here there is no major difference between them.

In Figure 5.5 and Figure 5.7 the performance for each LLM on each PDF files is presented instead of showing the average performance of the LLMs on all PDF files in Figure 5.2. As seen in In Figure 5.5 and Figure 5.7 there is a distinction of the LLMs performance on the aidonHaninterface File and especially on the Tables file were GPT-4-Turbo and GPT-4o is better. On the Tables file GPT-4o outperforms the other LLMs with a score 90% and on the aidonHaninterface file GPT-4-Turbo is better than GPT-4o with a score of 97%. Overall, these two LLMs performs best or among the best in each file. An interesting note is that mixtral-8x22B performs slightly better than the other LLMs on the Mixed1 file with a score of 85%.

6.2 Overall performance of the Parsing Tools/Methods

The average performance of the Parsing Tools with all LLMs can be seen in Figure 5.3, were the Multimodal llama_index_txt, llama_index_md and Unstructured Parsing Tools was the only ones scoring an average over 80% on all PDF files. The bottom three tools was the Rule Based PDFMiner_HTML, PDFMiner and PDFminer.six with scores under 65%, the Rule Based PyPDF and PyMuPDF Parsing Tools were slightly better than the bottom three with scores of 68%. In Figure 5.4 the same results are presented except here the scores of the Images file is removed. As seen in Figure 5.4 the llama_index_txt, llama_index_md and Unstructured tools are still among the top performers with scores of 88%, 88% and 86%. The PDFMiner_HTML is still the worst performing tool with a score of 68%. The PDFMiner and PDFminer.six tools now have scores of 80% and 81%, showcasing a performance better than the bottom three tools. The most interesting comparison is the PyMuPDF and PyPDF tools, in Figure 5.3 they had below average performance

compared with the other tools and in Figure 5.4, PyMuPDF scored 86% and PyPDF 87%. These two tools perform well on PDF files that does not contain complex images, PyPDF is even the third best performing tool with llama_index_txt and llama_index_md being slightly better.

Instead of presenting the Parsing Tools average performance on all PDF files, Figure 5.8 presents the performance for each Parsing Tool with all LLMs on each specific PDF file. Compared to the other Parsing Tools, the textract_corrected Parsing Tool performs above average in Figure 5.3 with a score of 77% and under average in Figure 5.4 also with a score of 77%. As shown in Figure 5.8 textract_corrected is the tool that performs the best on the Images file with good margin. This figure also shows the bad performance of the Rule Based Parsing Tools on the Images file, these are seen in the red colored upper right corner. This figure also shows the great performance of the Rule Based PyPDF and PyMuPDF Parsing Tools on all PDF files except the Images, were they performed as good as the Multimodal Unstructured Parsing Tool.

6.3 Performance on All PDFs

As presented in Figure 5.9, the Multitmodal llama_index_txt with GPT-4o performs the best with an overall average score of 90%. The second best with a score of 88% were the Deep Learning Based textract_raw_ocr combined with GPT-4-Turbo. The best overall performing Parsing Tools with all the LLMs are Unstructured, llama_index_txt and llama_index_md. GPT-4o and GPT-4-Turbo is the best overall performing LLMs with all the Parsing Tools.

The total prompt cost of the combo with llama_index_txt and GPT-4o were 0.5371 Dollar which is presented in Figure 5.19. The total prompt cost for the combo with textract_raw_ocr and GPT-4-Turbo were 0.9778 Dollar. Comparing the average score in Figure 5.9 with the prompt cost in Figure 5.19 the results is clear, the best performing Parsing Tools combined with the best performing LLMs is the most expensive options. According to both of these figures, a cost-efficient combination is the Multimodal llama_index_md and GPT-3.5-Turbo with a prompt cost of 0.047 Dollar and still maintains the performance with a score of 86%. The combination of the Multimodal Unstructured with llama38B had a score of 83% and a prompt cost of 0.0065 Dollar. This combo performs slightly worse than the combo of llama_index_md and GPT-3.5-Turbo, but it is more cost-efficient.

6.4 Performance on AidonHanInterface PDF

As seen in Figure 5.10, the Multimodal llama_index_md and llama_index_txt Parsing Tools performed the best with all LLMs with a score of 100%, the Rule Based PDFMiner Parsing Tool also had a score of 100% with all LLMs. The performance of the LLMs on the Parsing Tools are even, but GPT-4-Turbo performs the best with a score of 100% on eleven out of thirteen Parsing Tools and the worst performing LLM were mixtral-8x22B, with a score of 100% on only three of the Parsing Tools.

The combination of the Rule Based PyMuPDF and llama-3-8B scored 100% and had a total prompt cost of 0.0016 Dollar as seen in Figure 5.20. This combination is the among the best in scoring performance and is one of the most cost-efficient combination of Parsing Tools and LLMs, llama_index_txt with llama-3-8B had the same prompt cost. With

a score of 100% across all LLMs, the Rule Based PDFMiner in combination with llama-3-8B had a prompt cost of 0.0018 Dollar which also makes it a cost-efficient solution. llama_index_md with llama-3-8B had one of the cheapest prompt costs with 0.0015 Dollar and still maintained excellent performance with a score of 100%, which leads to this combination being the most cost-efficient.

These results showcases that the Rule Based Tools perform just as well as the Multimodal Tools and there is hardly any difference in the prompt costs between the tools. While combined with llama-3-8B the difference in prompt cost between PDFMiner and llama_index_md is 0.0003 Dollar, this demonstrates that both the Rule-Based and Multimodal tools are highly effective and cost-efficient in their performance on the Aidonhan-Interface PDF.

6.5 Performance on Images PDF

As shown in Figure 5.11, the Rule Based Parsing Tools don't perform at all on images with scores of 0% with all LLMs, except PDFminer with llama-3-70B and PDFMiner.six with GPT-4o were they scores 11%. The scores of 11% is likely due to the LLMs guessing the correct answers. Overall, there is low performance with no scores over 80% which it demonstrates how difficult it is to parse complex images. The Deep Learning Based textract_corrected Tools stands out from the mass where it did not have below 67% in score across the LLMs.

In terms of cost-efficiency one combination stands out from the mass. The combination of textract_corrected with llama-3-8B were among the best in performance with a score of 78% and as shown in Figure 5.21 the prompt cost of this combination were 0.0016 Dollar. Among the other tools, none achieved a 78% score except when combined with GPT-4-Turbo and GPT-4o. However, due to the high prompt cost, these combinations are not cost-efficient. Textract_corrected was the only tool capable of performing with a score of 78% with the other LLMs, which makes this tool necessary to obtain a cost-effective combination.

These results clearly demonstrates that a Deep Learning Based tool is necessary to handle PDF files containing complex images. Textract_corrected were superior compared to the Multimodal tools in both performance and cost-efficiency when combined with the LLMs, it even outperformed the other Deep Learning Based tools.

6.6 Performance on Mixed1 PDF

As demonstrated in Figure 5.12, the performance of the Parsing Tools and LLMs were equal on the Mixed1 file were three combinations of Parsing Tools and LLMs scored 100%, nine combinations scored only 67% and all the other combinations scored 83%. Out of the three best performing combinations the Deep Learning Based textract_corrected with mixtral-8x22B were the most cost-efficient with a prompt cost of 0.0076 Dollar as seen in Figure 5.22. The other two best performing combinations were the Multimodal llama_index_txt with GPT-4o which had a cost of 0.0485 Dollar and llama_index_txt with GPT-4-Turbo had a cost of 0.1016 Dollar. Among the combinations that scored 83% the Rule Based PyMuPDF with llama-3-8B had a prompt cost of 0.0007 Dollar, this combination had the same prompt cost as the Rule Based PyPDF with llama-3-8B.

On the Mixed1 file there is no superior Parsing Tool or combination of tool and LLM. With the overall small difference in performance across all tools and LLMs it is difficult to draw conclusions. The combinations of Parsing Tools and LLMs scoring 100% might be due to the LLMs guessing the correct answers. It seems illogical that they would achieve a perfect score with only one tool but not with others that enabled other LLMs to score 100%. For instance, `textract_corrected` with `mixtral-8x22B` scored 100%, while `llama_index_txt` with `mixtral-8x22B` scored 83%. Meanwhile, GPT-4o combined with the same tools scored 83% with `textract_corrected` and 100% with `llama_index_txt`. Because of this and the very similar performance among the tools, a more complex mixture of text, tables, and images would be needed for further analysis.

6.7 Performance on Tables PDF

In Figure 5.13 the GPT-4o and GPT-4-Turbo models outperforms the other LLMs with all the Parsing Tools. Overall, there is a large spread among the scores ranging from 11% to 100% among the combinations of Parsing Tools and LLMs. The Rule Based PyPDF with GPT-4o scored 89% and PyPDF with GPT-4-Turbo scored 100%, this is the same score as `llama_index_md` with GPT-4o(89%) and `llama_index_md` with GPT-4-Turbo(100%). This shows that a Rule Based Parsing Tool performs as well as a Multimodal Parsing Tool with these two LLMs. The best performing Parsing Tool were `llama_index_md` which had a score of 67% with only one LLM, with the other LLMs `llama_index_md` had a score of 89% or more. With a score of 89% and a prompt cost of 0.007 Dollar `llama_index_md` with GPT-3.5-Turbo was the most cost-efficient combination. The cost can be seen in Figure 5.23. The second most cost-efficient combination was `llama_index_txt` with GPT-3.5-Turbo which had a prompt cost of 0.008 Dollar and a score of 89%. While there is no major difference in prompt cost between these two combinations it clearly shows that a Multimodal tool combined with GPT-3.5-Turbo is the most cost-efficient on the tables file.

These results showcases that a Rule Based Parsing Tool combined with GPT-4o or GPT-4-Turbo has comparable performance to the Multimodal tools. However, GPT-4o and GPT-4-Turbo has the highest prompt cost making them not cost-efficient. The Multimodal tools had a significantly higher performance across all LLMs, which demonstrates their capability to interpret complex tables so it is understandable for all LLMs. This makes the Multimodal tools the superior choice in both performance and cost-efficiency where they can be combined with cheaper LLMs and still maintain high performance.

6.8 Performance on Texts PDF

Among the five best performing tools there is four Rule Based tools and one Multimodal tool which is demonstrated in Figure 5.14. The combination of the Multimodal Unstructured with `llama-3-8B` is among the top performing with a score of 100%. This is also the most cost-efficient combination with a prompt cost of 0.0009 Dollar as seen in Figure 5.24. Another cost-efficient combination is the Rule Based PyMuPDF with GPT-3.5-Turbo which had a prompt cost of 0.007 Dollar and a score of 100%. Unstructured also had a score of 100% with GPT-3.5-Turbo and was slightly more cost-efficient with a prompt cost of 0.0067 Dollar. Furthermore, the combination of the Rule Based PDFMinerSix with GPT-3.5-Turbo is also cost-efficient, with a score of 100% and a prompt cost of 0.0068 Dollar. Among these top performing combinations with scores of 100% there is

no major differences in prompt cost, except the prompt cost of Unstructured with llama-3-8B.

These results shows that the Rule Based Parsing Tools had higher performance across all LLMs. With four Rule Based tools being in the top five of performance, these tools demonstrates their capability of interpreting complex texts so its understandable for all LLMs and even does it significantly better than the Multimodal and Deep Learning Based tools.

6.9 Prompt Cost

The results indicates the choice of Parsing Tool does not affect the prompt cost, but rather the cost is affected by the choice of LLM. This was expected and is seen in Table 2.2, but the results strengthens it further. As seen in Figure 5.19 the difference of the prompt cost among the tools on the same LLM is small, for example the distinction of prompt cost between llama_index_txt and PyMuPDF with llama-3-8B is 0.0018 Dollar and the difference in prompt cost between llama-3-8B and GPT-3.5-Turbo with PyMuPDF is 0.0349 Dollar. One of the Parsing Tools actually affects the prompt cost, as presented in Figure 5.15 the Rule Based PDFMiner_HTML tools significantly affects the prompt cost compared to the others. This could be because when PDFMiner_HTML parses the PDF files, it saves the interpreted content in an HTML file, which can be much more difficult and complicated for LLMs to understand. As a result, the prompt cost becomes significantly higher which is shown in Figure 5.18.

6.10 Questions on Aidonhaninterface PDF

Over all the the tools and LLM scored well on the Aidon file. One thing that stands out is the question: "What does the sent data List from 3-phase ESD begin and end with?", with this question all the tesseract variations and Unstructured OCR had issues. All the LLMs failed with this tool except GPT-4 Turbo which manged in to get the right answer with this tools in 50% of them.

6.11 Questions on Images PDF

For the question "How many thinks the government is doing too little to protect air quality?" Only llama index that manage to get that question right and with all the LLMS, thus showing it was the only one to interpret it right and in a good way since all the LLMs can understand it. But the for the next question "How many thinks the government is doing too much to protect open lands in national parks?" that is from the same image it failed with all the LLMs and only tesseract corrected and filtered manage to get it right with all the LLMs. This was a bit surprising that they only got one question right on the same type of data from the same graph. So, we looked at the parsed data to if we cloud figure out why this happened. We noticed that llama index got the 2 first parts of the graph right but missed the last part containing the "Too much" asked in question it failed on. For tesseract the files did not contain the data asked by the second question but the one for the first! How could it get the right answer? We where very confused by this. Later we remember that we have noticed that the files produced by tesseract do not always look the same, there is some randomness in how well it can extract the data from the PDF files. The tesseract pared files we where looking at was not the same one used, every time this

paring part of the program is used it overwrites the old parsed files. It might been a good idea to keep the original files used to in the test, but if we did that we have might missed this valuable insight.

6.12 Questions on Mixed1 PDF

For this file there where two questions that did not a full score by all the tools and LLMs. It was the question "What is the brand on the beer bottle in Figure 1: Beer?" It was only two tools that got this right, or at least it looked like that at first. When we looked at the individual answer by the LLMs we found that that one of the was a false positive by the LLM scorer. So it was acutely only one of the tools that manege to get the question right and that was llama index txt. What was a bit surprising was that it was only one of the llama index that got it right. When we looked at the parsed file we could see that only the txt file had manged to get the text in the image parsed. But despite that it have manege to extract the name of the beer it was only GPT 4 Turbo and omni that manged to understand the content in the pared file. When looking and comparing the two llama index files we noticed that the txt versions tries to keep the original format of the file by using spaces to move the text to look like the original file. Meanwhile the markdown version do not to this but instead converts using markdown features like headers and tables to retain some of the original formatting. The other questions that some of them failed on was "When was Sweden industrialized and urbanized?". There all of them except Unstructured hi res and Unstructured OCR got a full score. For this questions the answer in the text was split by an image. So it seems that the ocr only versions of unstructured can not handle this appropriate since all of the model failed on at least one of them.

6.13 Questions on Tables PDF

For this file it seems like the LLMs is the more important factor for the result. The two best LLMs GPT-4 Turbo and omni manages to get all the questions right with several of the parsing tools. While the rest did not mange a full score with any of the parsing tools. The multimodal tools seems to handle the tables better than the rest. With to of the rule based also manage to get a full score with the best LLMs. This also indicates that the LLMs is more important for the understanding of tables. Here we also had a question that required some processing of the data "How many players scored over 105 points?". This question it was GPT-4 Turbo and omni that got the best score for this, followed by mixtral. This indicates that for questions that need more analyzing and understanding of the data it is needed to use the higher performing LLMs. The rest of the LLM have a similar score overall. For the questions "What is the date to Rome according to the table?" only GPT 3.5 have a large issue go get the right answer performing significantly worse. For the question "Which player had the most wins and what was his save percentage?" The tesseraact tools and the llama LLMs struggled with.

6.14 Questions on Texts PDF

In the PDF text file, most of the questions were answered correctly by all the tools and LLMs. However, the two most challenging questions was about the header and footer of the file. We could not identify any trend that made it possible to answer these questions

correctly using either the parsing tools or LLMs. For the question, "When did Meta release Llama-3?" both tesseract and Unstructured OCR struggled. The text containing the answer was split by a picture, highlighting a issue with OCR-based tools when text is divided into different columns.

6.15 Conclusion of Analysis

In conclusion, a Multimodal Parsing Tool is the preferable choice in the overall performance on all PDF Files. As seen in Figure 5.9, the overall performance of the Multimodal Parsing Tools is the best. The Multimodal tools also performs best with all LLMs, it means that these tools manage to parse the PDF files in way that all LLMs can understand. On the Images PDF file a Deep Learning Based Parsing Tool is the superior option were these tools outperforms the rest even with smaller LLMs which is demonstrated in Figure 5.11. When it comes to PDF files containing only text and tables the Rule Based Parsing Tools performs equal to the Multimodal Tools.

7 Discussion

In this chapter the responses to the research questions (RQs), observations and the broader implications of findings are explored. Additionally, this chapter addresses the potential for larger scaled tests, the costs associated with different parsing tools, and the rapid evolution of LLMs.

7.1 RQ 1

RQ 1, How do the different parsing tools/methods handle the varying complexity of the PDFs, what are the benefits and limitations of the methods?

Concerning RQ 1, the results indicate notable differences in the performance of the Parsing Tools/Methods. The short answer is that the multimodal tools are the most versatile and effective for a range of PDF formats, deep learning-based tools excel specifically with complex images, and rule-based tools are best for text and table-heavy PDFs excluding images.

As presented in the Analysis, the multimodal Parsing Tools performed best overall with all LLMs on all PDF files. The benefits of utilizing the multimodal tools is their capability to handle the diversity of PDF files presented to them, these tools can manage PDF files containing different complexities and formats of images, tables and texts in a effective way. One limitation of the multimodal Tools is the that they were outperformed by one of the Deep Learning Based Tools when parsing the PDF file containing various complicated images and scanned images. On images the results demonstrates how one of the Deep Learning Based Tools performed significantly better than the multimodal tools with all LLMs.

The Deep Learning Based Tools performed averagely overall with the LLMs on all PDF files, were they performed neither poorly nor well. These tools were not superior in handling the PDF files containing images of different complexities, which was not expected because in theory these tools should be the obvious choice on images. With that said, the The Deep Learning Based `textextract_corrected` Parsing Tool performed significantly better than every other method on the images. Therefore, the benefit of this method is in handling PDFs that include various complex images, if the method is used in the right tool. The limitations of the deep learning-based method is the mediocre performance on all the PDF files, leading to the other methods being a better choice.

The overall performance of the Rule Based Tools with the LLMs on all PDF files were the worst of all the methods, but when the results on the file containing images were excluded, these tools were among the best with a performance comparable to the multimodal tools. One clearly limitation of the Rule Based Method is the incapability to handle images at all. The benefits are clear, the Rule Based Tools are capable of managing larger and more complex PDF files containing tables and text in different formats. On these type of files the Rule Based Tools performed equally good as the multimodal tools, however on files containing only complex tables they were outperformed by the multimodal tools and on files containing only text they were significantly better than the other methods.

7.2 RQ 2

RQ 2, How do the different LLMs perform on the varying complexity of the parsed PDFs, what are the strengths and weaknesses of each model?

Regarding RQ 2, the overall performance of GPT-4-Turbo and GPT-4o were superior compared to the other four LLMs. The strengths of GPT-4-Turbo and GPT-4o are clear, they are superior in handling larger and more complex parsed files, even when the files includes a mixture of tables and texts. The weakness of GPT-4-Turbo and GPT-4o are their performance on files containing images, while these two still are the best on images there is no significantly difference in performance compared to the other LLMs.

The four other LLMs, GPT-3.5-Turbo, llama-3-70B, llama-3-8B and mixtral-8x22B were equal in overall performance. These four LLMs showed capability's in handling parsed files containing different formats of texts and even files containing a mixture of text, tables and images. The strengths of GPT-3.5-Turbo, llama-3-70B, llama-3-8B and mixtral8x22B was their comparable performance on images where they were equally good or slightly worse compared to GPT-4-Turbo and GPT-4o. The weaknesses of GPT-3.5-Turbo, llama-3-70B, llama-3-8B and mixtral-8x22B are clear, they lack the ability to effectively handle files containing a variety of complex tables, where their performance is significantly worse than GPT-4-Turbo and GPT-4o.

7.3 RQ 3

RQ 3, How do the different parsing methods and LLMs compare in terms of cost-efficiency?

For the final RQ 3, the results show substantial differences in the cost-efficiency of the tools and LLMs. As anticipated, the two industry-leading models, GPT-4-Turbo and GPT-4-O achieved the highest performance both overall and for each parsing tool, but they are also the most expensive. As the results demonstrates, the difference for the prompt costs between the tools on the same LLM is relatively small and this shows that the difference is dependable on the LLMs rather than the Parsing Tools. The multimodal method impacts the prompt cost slightly more than the Rule Based Method, with a difference of only 0.0018 Dollar when compared using Llama3-8B with the multimodal llama_index_tx and Rule-Based PyMuPDF. Despite this minor cost difference, the superior parsing performance of the multimodal method across all PDF files in this study makes it the most cost-efficient option.

Regarding the LLMs, GPT-4-Turbo and GPT-4o were the best performers. However, their significantly higher prompt costs make them less cost-efficient. Mixtral8x22B had the poorest performance and was the third most expensive in terms of prompt cost, making it inefficient. Llama3-8B, despite being the cheapest, only slightly outperformed Mixtral8x22B, diminishing its cost-efficiency. GPT-3.5-Turbo and Llama3-70B had equal performance, which was not far behind GPT-4o. Although their prompt costs were also equal, GPT-3.5-Turbo's slightly lower prompt cost makes it the better option in terms of overall cost-efficiency.

As a summary of the cost-efficiency of the parsing methods and LLMs, a combination of the multimodal method with GPT-3.5-Turbo, which has the second cheapest prompt

cost and maintains good performance, along with the effective parsing performance of the multimodal Method, would result in a cost-efficient solution across the PDF files used in this study.

7.4 Related Works

Demiao Lin’s Investigation on PDF Parsing Accuracy: Lin’s research emphasizes the impact of PDF parsing accuracy on professional knowledge-based QA systems, demonstrating the limitations of current methods that are rule-based. Our study extends this by demonstrating that multimodal methods outperform traditional rule-based approaches in handling mixed PDF content, including complex images and tables. This aligns with Lin’s conclusions regarding the importance of accurate PDF parsing but provides a broader evaluation across different types of content.

Naihao Deng et al.’s Study on multimodal LLMs: Deng’s work on the effectiveness of multimodal LLMs in interpreting tabular data using different prompting strategies presents a analysis across various benchmarks. Our findings acknowledge their results, showing that multimodal methods can enhance parsing accuracy, especially in handling mixed content formats. However, our study also identifies specific scenarios where deep learning-based methods surpass multimodal tools, particularly for image PDFs, thus adding nuance to Deng’s broader conclusions.

7.5 Observations

One notable observation is that there wasn’t a single question that couldn’t be answered using a combination of parsing tools and an LLM. However, several questions could only be addressed by one or very few tools, while the others failed to provide accurate answers. This highlights the fact that no single parsing tool can effectively handle the vast variety of PDFs. We didn’t find a clear indication of the types of questions each tool would perform best on, indicating that it’s difficult to predict how well a specific part of a PDF will be parsed. This raises the question of whether a combination of tools might achieve a noticeably better score than any single tool alone. We observed that the LLM often indicates when it lacks the correct information from the parsing tool to answer a question. By employing multiple tools, it might be possible to use both versions of the parsed PDF, leading to improved scores. This method could potentially allow achieving the same score as a more expensive LLM using a cheaper LLM, thereby reducing costs while maintaining performance.

To evaluate the feasibility of combining multiple tools, we conducted a primitive simulation using results from our tests. We selected two parsing tools and the least expensive model (Textract Corrected, Llama Index TXT, and Llama-3-8b-Instruct). For each question, we recorded the best score from either parsing tool, simulating the LLM having access to both parsed versions and selecting the best one. This approach increased the score from the previous best of 83% to 86%, demonstrating that using multiple tools can indeed improve performance. Additionally, we applied this method to the best-performing LLM, which saw its score increase from 90% to 95%. While this result might not be directly applicable in a real-world test, since the LLM might not inherently recognize which parsed data contains the best information from the PDF, it still suggests that combining multiple tools could yield better results and that there might be even better combinations within

our results.

Another interesting discovery made along the way of the experiment is that the Parsing Tool that produces a file that looks the best with the human eye is not necessary the best for the LLM. The parsed file from tesseract looks really good, well structured with the content but when the LLMs processed the file it did not get as high score as some of the parsed files that looks bad with the human eye. It is a big difference in how we(humans) interpret the parsed files and how the LLMs do it. The parsed file from looks bad structured and unorganized but the LLMs can interpret it just fine. An interesting discovery for sure.

A surprising observation is the Parsing Methods and LLMs performance on the Aidon-Haninterface PDF where several tools managed to interpret this in way the LLMs understand with several score of 100%. With the complex structure and format of this lengthy file containing complex tables, text and Images it was anticipated that the performance would be significantly lower than the results presents.

7.6 Larger Scaled Test

In a real-world application, the performance differences among the cheaper models might become more pronounced. The questions we posed in our tests were straightforward and easily extracted from the PDFs. However, in practical applications, the questions asked might be more complex, requiring not only the retrieval of relevant data but also a deeper understanding and interpretation. This increased complexity could more clearly highlight the strengths and weaknesses of different models. It is challenging to determine whether the small variance in results is due to the similar performance of all tools and LLMs or if larger-scale testing is required to identify significant differences. The current tests provide valuable insights but may not capture the full range of capabilities and limitations of each tool and model. Expanding the scale of testing could help to uncover more nuanced differences and inform better choices for specific applications.

7.7 Comparing Cost of Parsing Tools

Regarding the Parsing Tools/Methods, the tool with the best performance is not necessarily the cheapest. When it comes to comparing the cost of these parsing tools, the analysis becomes complex due to the diverse nature of their operational requirements and associated expenses. It is not possible to make a fair comparison of the different tools tested in this study. Since some of them operate locally on a computer, some use an LLM that can operate locally or by using an external API and some are only accessed via external API. The parsing tools that operates locally, like the Rule Based tools leads to cost related to the electricity consumption of the computer. This can be significant, especially for tools that require intensive processing power over extended periods. Some of the multi-modal tools that utilizes an LLM are possible to operate locally, these tools would require substantial computational resources. Running such models effectively demands a powerful computer equipped with high-performance hardware, such as advanced GPUs, ample RAM, and fast storage solutions. This not only involves a considerable initial investment in hardware but also incurs ongoing operational and maintenance expenses. The multi-modal parsing tools utilizes APIs, which involves a direct financial cost based on usage.

These costs can accumulate quickly, particularly with high-frequency or large-scale data parsing needs. The API-based tools often offer convenience and ease of integration but come at the price of ongoing subscription or pay-per-use fees. These costs are also computed differently for the tools, some of them are calculated based on the number of pages parsed and some on the number of tokens.

7.8 Evolution of LLMs

The lifespan of the results regarding the LLMs will probably be outdated when this study is published. New LLMs are released continuously and these are faster, have higher performance and above all they are significantly cheaper. During the time working with this study two major LLMs were released that outperform their older predecessor and there are probably a few more that we don't know about, the two we do know about are Llama3 and GPT-4o. The Llama3 model with 8B and 70B parameters was released on April 18(2024) by Meta and presented a major leap over its predecessor Llama2 by establishing a new state-of-the-art performance at these scales. This was possible thanks to improvements in pre-training and post-training, these improvements reduced false refusal rates and increased the diversity in model responses and improved alignment. The GPT-4o model was released by OpenAI on May 13(2024) and it is trained on a new single end-to-end model, which means that all inputs and outputs are processed by the same neural network. GPT-4o achieved similar performance to its predecessor GPT-4-Turbo, in text, reasoning, and coding intelligence, while making unprecedented advancements in multilingual, audio, and vision capabilities. According to OpenAI, they are still only scratching the surface of the potential of a model that combines all these modalities. GPT-4o matches GPT-4-Turbo in performance but is significantly faster and 50% cheaper in both input and output costs.

These two LLMs showcase how rapidly the evolution of LLMs is advancing in every way. The newly released models demonstrate significant improvements in multiple dimensions, including language comprehension, context retention, and generative capabilities. One of the most notable advancements is their enhanced multilingual proficiency. These LLMs can now handle a broader range of languages with greater accuracy and fluency, making them invaluable tools with their ability to understand and generate text in various languages. GPT-4o exhibits superior vision processing abilities, this capability allows it to analyze and describe images and videos with remarkable detail, opening up new possibilities in data processing. The integration of these advanced capabilities into a single model marks a significant milestone in the development of artificial intelligence. As these LLMs continue to evolve, they will undoubtedly bring changes across various sectors, driving innovation and efficiency in ways previously thought impossible. Despite these impressive advancements, it is important to recognize that we are still at the early stages of fully understanding the potential and limitations of these models. Who knows, maybe OpenAI releases GPT-5 in a month, making our results of GPT-4o outdated.

8 Conclusion

In conclusion, this degree project investigates various methods and tools for parsing PDF documents into LLMs. Our study focused on evaluating the performance and cost-efficiency of different parsing methods and LLMs when handling diverse and complex PDF formats. This project underscores the importance of selecting the right combination of tools and models to achieve both high performance and cost-efficiency in document processing tasks. By balancing these factors, organizations can enhance their data management practices, leading to more efficient operations and reduced costs. Significant differences were found in the performance of various parsing tools and methods. Multimodal tools consistently performed well across different types of PDF files, including those with complex layouts and mixed content (text, images, tables). While multimodal methods were generally effective, deep learning-based tools excelled particularly in handling images. Rule-based tools showed strength in parsing text and tables but struggled with images. Among the LLMs tested, GPT-4-Turbo and GPT-4o demonstrated superior performance, particularly with complex and varied data. Other LLMs like GPT-3.5-Turbo and various Llama models performed well but had limitations with complex tables and images.

While the study focused on PDF parsing, the methodologies and findings can be applied to other areas requiring efficient data extraction and processing from complex documents. The insights gained from comparing different LLMs and parsing tools can guide future developments in document processing technologies. Parsing PDFs still presents significant challenges, and our results indicate that no single method excels in all aspects. While some good solutions exist, challenges remain due to the wide diversity of PDF formats, complexities, sizes, and contextual layouts. The results of this study highlight these challenges and illustrate the difficulties in finding a universally effective approach. A promising method is the use of a multimodal Method combined with a Deep Learning-Based Method to parse PDFs. This research contributes to the broader field of natural language processing and information management, offering practical guidance on the most effective methods for PDF parsing in real-world applications. Future work could explore further advancements in LLMs and parsing technologies, aiming to enhance their capabilities and address the remaining challenges identified in this study.

8.1 Future work

More Complex Mixed PDFs, the results demonstrates a extremely equal performance among the Parsing Tools/Methods and LLMs on the Mixed1 PDF where a clear conclusion was difficult to draw. This file contains a relatively simple mixture of text, tables and image in different formats that break off each other. As noted in the analysis of the tools and LLMs performance on this file, a more complex mixture of text, tables, and images would be required for further investigation. This approach could lead to more diverse results, allowing for a clearer conclusion and highlighting individual tools with superior performance.

LLMs Analysis Capability, The questions in this study are simple and straightforward. However, in a real-world application where more complex questions are posed, LLMs would need to perform a more extensive analysis of the data. In such scenarios, there might be more significant differences in performance between the various LLMs. Therefore, it is interesting to test with more complicated questions that require the LLMs to

analyze the data.

Combine Parsing Tools, Investigate the potential of combining multiple parsing tools to leverage their individual strengths. For instance, integrating the multimodal llama_index for text and table extraction and the Deep Learning Based Textract_corrected for images could improve overall parsing accuracy.

Focus on best Performing Parsing Method, Based on the results of this study, the multimodal Method stands out as the best overall parsing approach. Given its performance, it is valuable to focus on experimenting with these tools exclusively. By comparing various multimodal tools, which are frequently released and updated, it would lead to additional insights into their effectiveness in PDF parsing.

Comparing Parsing Tools Costs, Comparing the costs associated with different parsing tools provides a deeper and more valuable analysis of their true value and efficiency. While costs vary significantly across different parsing methods, focusing solely on multimodal tools allows for a more precise cost-analysis. This detailed comparison between multimodal tools will highlight the financial and operational aspects, providing insights into which tool offers the best balance of cost and performance for specific use cases.

References

- [1] P. H. Silverplate and D. Jarvis, “Challenges in data management,” 2022, accessed: 2024-05-20. [Online]. Available: <https://www2.deloitte.com/us/en/insights/industry/technology/challenges-in-data-management.html>
- [2] M. Pandey, M. Arora, S. Arora, C. Goyal, V. K. Gera, and H. Yadav, “Ai-based integrated approach for the development of intelligent document management system (idms),” 2023, accessed: 2024-04-12. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050923021324>
- [3] L. Stephen Wolfram, “What is chatgpt doing ... and why does it work?” 2023, accessed: 2024-04-18. [Online]. Available: <https://writings.stephenwolfram.com/2023/02/what-is-chatgpt-doing-and-why-does-it-work/>
- [4] A. K. Kar, P. S. Varsha, and S. Rajan, “Unravelling the impact of generative artificial intelligence (gai) in industrial applications: A review of scientific and grey literature,” 2023, accessed: 2024-04-12. [Online]. Available: <https://link.springer.com/article/10.1007/s40171-023-00356-x>
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021.
- [6] Nehal, “Introduction to data extraction from pdfs: Tools and techniques,” 2024, accessed: 2024-05-20. [Online]. Available: <https://www.promptcloud.com/blog/introduction-to-data-extraction-from-pdfs-tools-and-techniques/>.
- [7] G. Smith, M. Bateman, R. Gillet, and E. Thanisch, “Environmental impact of large language models,” 2023, accessed 2024-04-02. [Online]. Available: <https://www.cutter.com/article/environmental-impact-large-language-models>
- [8] D. Lin, “Revolutionizing retrieval-augmented generation with enhanced pdf structure recognition,” *arXiv preprint arXiv:2401.12599*, 2024, accessed: 2024-03-26. [Online]. Available: <https://arxiv.org/pdf/2401.12599.pdf>
- [9] N. Deng, Z. Sun, R. He, A. Sikkam, Y. Chen, L. Ma, Y. Zhang, and R. Mihalcea, “Tables as images? exploring the strengths and limitations of llms on multimodal representations of tabular data,” *arXiv preprint arXiv:2402.12424*, 2024, accessed: 2024-03-27. [Online]. Available: <https://arxiv.org/pdf/2402.12424.pdf>
- [10] J. Gavilanes, Y. Bozhilov, U. Dodeja, G. Valtas, and A. Badrajan, “Use of LLM for Methods of Information Retrieval,” Enschede, 2023, p.O. Box 217, 7500AE Enschede, The Netherlands.
- [11] M. Korpela, “Extracting data from pdfs: No silver bullet,” 2024, accessed: 2024-04-19. [Online]. Available: <https://robocorp.com/blog/extracting-data-from-pdfs-no-silver-bullet>
- [12] Adobe, “Everything you need to know about the pdf.” 2024, accessed: 2024-04-18. [Online]. Available: <https://www.adobe.com/acrobat/about-adobe-pdf.html>

- [13] S. AI, “Challenges in text extraction from pdfs,” 2023, accessed: 2024-04-18. [Online]. Available: <https://medium.com/@soffosdotai/challenges-in-text-extraction-from-pdfs-62037c854f00>
- [14] IBM, “What are large language models (llms)?” -, accessed: 2024-04-29. [Online]. Available: <https://www.ibm.com/topics/large-language-models>
- [15] OpenAI, “Gpt-3.5 turbo fine-tuning and api updates,” 2024, accessed: 2024-05-14. [Online]. Available: <https://openai.com/index/gpt-3-5-turbo-fine-tuning-and-api-updates/>
- [16] —, “Models,” 2024, accessed: 2024-05-14. [Online]. Available: <https://platform.openai.com/docs/models>
- [17] —, “Gpt-4 is openai’s most advanced system, producing safer and more useful responses,” 2024, accessed: 2024-04-30. [Online]. Available: <https://openai.com/gpt-4>
- [18] —, “Gpt-4,” 2023, accessed: 2024-04-30. [Online]. Available: <https://openai.com/research/gpt-4>
- [19] —, “Hello gpt-4o,” 2024, accessed: 2024-05-14. [Online]. Available: <https://openai.com/index/hello-gpt-4o/>
- [20] Meta, “Introducing meta llama 3: The most capable openly available llm to date,” 2024, accessed: 2024-05-14. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3/>
- [21] M. AI, “Cheaper, better, faster, stronger,” 2024, accessed: 2024-05-14. [Online]. Available: <https://mistral.ai/news/mixtral-8x22b/>
- [22] TensorOps, “Understanding the cost of large language models (llms),” 2023, accessed: 2024-05-19. [Online]. Available: <https://www.tensorops.ai/post/understanding-the-cost-of-large-language-models-llms>
- [23] OpenAI, “What are tokens and how to count them,” 2021, accessed: 2024-05-19. [Online]. Available: <https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>
- [24] TechTarget, “Context window,” n.d., accessed: 2024-05-19. [Online]. Available: <https://www.techtarget.com/whatis/definition/context-window>
- [25] Databricks Inc., “What is retrieval augmented generation, or rag?” 2024, accessed: 2024-04-09. [Online]. Available: <https://www.databricks.com/glossary/retrieval-augmented-generation-rag>
- [26] R. Merrit, “What is retrieval augmented generation, aka rag?” 2023, accessed: 2024-04-09. [Online]. Available: <https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/>
- [27] G. peretz, “Openrouter + langchain: Leverage opensource models without the ops hassle,” 2024, accessed: 2024-04-30. [Online]. Available: <https://medium.com/@gal.peretz/openrouter-langchain-leverage-opensource-models-without-the-ops-hassle-9ffbf0016da7>

- [28] PyPDF Contributors, “Pypdf: A python library for working with pdf documents,” <https://github.com/py-pdf/pypdf>, 2024, accessed: 2024-04-16.
- [29] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, “Nougat: Neural optical understanding for academic documents,” 2023.
- [30] R. Smith, “An overview of the tesseract ocr engine,” in *ICDAR ’07: Proceedings of the Ninth International Conference on Document Analysis and Recognition*. Washington, DC, USA: IEEE Computer Society, 2007, pp. 629–633. [Online]. Available: <https://storage.googleapis.com/pub-tools-public-publication-data/pdf/33418.pdf>
- [31] F. Jiang, K. Wang, and H. Li, “Bridging research and readers: A multi-modal automated academic papers interpretation system,” 2024.
- [32] PyPDF2Contributors, “Pypdf2 documentation,” [Online]. Available: <https://pypdf.readthedocs.io/en/latest/index.html>, 2024, accessed on: Apr. 18, 2024.
- [33] —, “Pypdf2 parsing documentation,” [Online]. Available: <https://pypdf.readthedocs.io/en/latest/dev/pypdf-parsing.html>, 2024, accessed on: Apr. 18, 2024.
- [34] —, “Extract text pypdf2 documentation,” [Online]. Available: <https://pypdf.readthedocs.io/en/latest/user/extract-text.html>, 2024, accessed on: Apr. 18, 2024.
- [35] A. S. Inc., “Pymupdf documentation,” Read the Docs, 2024, accessed: 2024-04-22. [Online]. Available: <https://pymupdf.readthedocs.io/en/latest/index.html>
- [36] R. Watts, *MuPDF Explored*. Artifex Software Inc., 2022.
- [37] Unstructured-IO, “Unstructured documentation,” <https://unstructured-io.github.io/unstructured/>, 2023, accessed: 2024-04-22.
- [38] —, “unstructured,” <https://github.com/Unstructured-IO/unstructured>, 2023, accessed: 2024-04-23.
- [39] Y. Shinyama, P. Guglielmetti, and P. Marsman, “Welcome to pdfminer.six’s documentation!” 2023, accessed: 2024-04-22. [Online]. Available: https://pdfminersix.readthedocs.io/en/latest/topic/converting_pdf_to_text.html#layout-analysis-algorithm
- [40] LlamaParse, “Llamaparse: Advanced document parsing tool,” 2024, accessed: 2024-05-21. [Online]. Available: <https://llama-2.ai/llamaparse/>
- [41] J. Emanuel, “llama2_aided_tesseract,” 2023, accessed: 2024-04-22. [Online]. Available: https://github.com/Dicklesworthstone/llama2_aided_tesseract
- [42] Y. Liu, Z. Li, B. Yang, C. Li, X. Yin, C. lin Liu, L. Jin, and X. Bai, “On the hidden mystery of ocr in large multimodal models,” 2024.

A Appendix 1

The Source Code and data can be found here: github.com/PDF-Parsing-Unveiling-the-Most-Effective-Method